

Taking Orbits to the Limit:

Inference Replaces Memory in Operator-Space Neural Networks

Sirsh

April 2026 — Working Draft

Abstract

A transformer reads a sequence by transforming states: L layers of attention and feed-forward computation, each materialising an intermediate representation that the next layer consumes. We ask whether the coupling between tokens—which positions attend to which—can be separated from the response to that coupling and computed algebraically, without materialising intermediate states.

We begin with an empirical observation from the Orbital Response Network: a single shared coupling operator $\mathbf{M}$ crystallises during training to a stable low-rank structure and transfers across tasks. We formalise this as the question: does the state trajectory of an L -layer network lie in the Krylov subspace of $\mathbf{M}$? If so, a polynomial of $\mathbf{M}$ recovers the trajectory and intermediate states are redundant.

The answer is nuanced. The Krylov subspace recovers 36% of the trajectory. Per-layer analysis reveals a clean split: early layers (1–4) are 96% linear in $\mathbf{M}$ —the operator dominates. Deep layers (5–8) are 50% nonlinear—the feedforward network does real work. A polynomial alone cannot replace L layers. But the polynomial captures the *coupling structure*—the relational grammar—while the FFN handles *content*: which specific tokens fill the structural slots.

This decomposition motivates a new architecture: **asymmetric geometric attention**. The coupling operator $\mathbf{M} = AB^\top$ factors every polynomial into query-side (A) and key-side (B) projections through a reduced inner geometry $G = B^\top A$. Input-dependent coefficients select how deeply each token engages with G ’s spectral structure—a template selector, not a complexity estimator. The polynomial operates in r -dimensional space (cheap), the asymmetry is built in ($A \neq B$), and re-coupling after nonlinear response lets the model see what the FFN discovered before routing again through the same geometry.

Experiments show: coupling is the bottleneck, not response depth. Two FFN layers match four. The value is not in deeper response but in re-coupling—evaluating the geometry again after transformation. This is the ORN’s coupling-response decomposition taken to its limit: the coupling IS the geometry, and the geometry is what you learn.

1 Inference Replaces Memory

The word “compression” pervades modern sequence modelling. Mamba “compresses” a sequence into a hidden state [2]. S4 “memorises” a history via a state-space model [1]. Linear attention “accumulates” key-value pairs into a fast-weight matrix [3]. The metaphor is always the same: the network receives information, stores it in a state, and later retrieves it.

But compression implies that the information existed first and was then reduced. That the state *contains* the sequence, in degraded form. That the network’s job is to hold on to what it has seen.

We propose a different framing. The network does not hold information. It holds a *rule*—a relational grammar encoded as an operator $\mathbf{M}$ —from which the appropriate response to any input can be *inferred*. The coupling pattern at position 50 is not a memory of positions 1–49. It is a consequence of applying the grammar to whatever appears at positions 1–49. The

operator did not see those tokens and store a summary. It defines a geometry, and the geometry determines what must follow.

This is the difference between a lookup table and a function. A lookup table stores entries. A function generates them. The function never “had” the entries—it computes them on demand from its rule. A trained operator $\mathbf{M}$ is a function over token relationships, not a table of stored states.

Why this matters for architecture. If inference replaces memory, then the sequential state-to-state computation in every existing architecture is doing unnecessary work. Each layer materialises an intermediate state $\mathbf{h}_l$, but that state is largely determined by $\mathbf{M}$ applied to the input—the intermediate values are not independent contributions, they are waypoints along an orbit that the operator defines.

An architecture built on this principle would not compute states layer by layer. It would compose the operator with itself—algebraically, in operator space—and produce a state only when one is needed for readout. The depth of computation would be a property of the operator (its spectral structure, its polynomial degree), not a count of sequential passes through a layer.

2 The State-Space Status Quo

To be precise about what we propose to replace:

Every transformer, every RNN, every residual network computes as a state machine:

$$\mathbf{h}_0 \xrightarrow{L_1} \mathbf{h}_1 \xrightarrow{L_2} \mathbf{h}_2 \xrightarrow{L_3} \dots \xrightarrow{L_L} \mathbf{h}_L \quad (1)$$

Each intermediate state $\mathbf{h}_l$ is fully materialised. The depth of computation is exactly L , regardless of input difficulty.

Weight-tied architectures (Universal Transformer, ALBERT, ORN) share parameters across layers: $L_1 = L_2 = \dots = L_L$. This reduces the parameter count but not the computational pattern. The ORN shares a coupling operator $\mathbf{M} = AB^\top$ across all layers, but **it is still a state-space machine**. $\mathbf{M}$ is applied to states, not composed with itself. There is no deferred materialisation, no algebraic composition, no semigroup computation. The orbit through $\mathbf{M}$ is a metaphor for the state trajectory, not a computational reality.

The question is whether the metaphor can be made literal.

3 Operator-Space Computation

We define the alternative precisely:

Definition 1 (Operator-space computation). A function $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ computed as $f = g \circ T$ where T is an operator constructed from learned parameters *without materialising intermediate states*, and g is a nonlinear readout applied once to $T(\mathbf{h}_0)$.

Contrast with the state-space definition: $f = L_L \circ \dots \circ L_1$ with all $\mathbf{h}_l = L_l(\mathbf{h}_{l-1})$ materialised.

If $\mathbf{M}$ generates a semigroup $T(t) = \exp(t\mathbf{M})$, then operator-space computation applies $T(L)$ once. For a purely linear network, the two are equivalent:

$$\mathbf{h}_L = \mathbf{M}^L \mathbf{h}_0 \quad (\text{state space}) \quad = \quad \exp(L \log \mathbf{M}) \mathbf{h}_0 \quad (\text{operator space}) \quad (2)$$

The question becomes non-trivial when nonlinearity is present. In the standard ORN, each layer applies:

$$\mathbf{h}_{l+1} = \mathbf{h}_l + \text{Attn}_{\mathbf{M}}(\mathbf{h}_l) + \text{FFN}(\mathbf{h}_l) + \sigma(g(\mathbf{h}_l)) \cdot \text{corr}(\mathbf{h}_l) \quad (3)$$

The FFN and corrections introduce nonlinearity at every layer. Can this be decomposed into a dominant operator plus a nonlinear readout? The next section states the conjecture precisely.

4 The Central Conjecture

4.1 What We Are Trying to Build

We are not optimising loss on TinyStories. We are trying to build a new type of circuit: one where an operator $T(\mathbf{M})$ —a learned functional of the coupling geometry $\mathbf{M}$ —encodes a *relational world model*, and a nonlinear readout (FFN) interprets that model to produce specific outputs.

The relational world model is not a set of facts. It is a geometry of relationships: which concepts relate to which, through what templates, at what nesting depth. “Vindicate” activates a three-party judgment template. “Despite” activates an expectation-violation template. The operator encodes these templates in its spectral structure. The FFN’s job is to interpret the activated templates in context—to fill the slots, resolve the references, produce the tokens.

The operator builds a simulacrum of relational structure. The FFN wraps it in specifics.

4.2 The Structural Invariance Conjecture

Conjecture 1 (Structural Invariance). *Natural language (and possibly other structured data) possesses reusable relational invariants that repeat across contexts and levels of abstraction. These invariants:*

1. *Are discoverable by gradient descent from raw data.*
2. *Can be decoupled from the context-specific processing that uses them.*
3. *Are significant—they carry enough of the data’s structure that context-specific processing reduces to slot-filling given the invariants.*

This is a claim about language, not about any specific parameter matrix. SharedM and SharedFFN are one possible implementation. The crystallisation, transfer, and parameter sharing results are evidence that such invariants exist (Part 1), are separable (Part 2), and carry substantial information (Part 3—though the evidence here is weaker).

If Conjecture 1 only held, it would motivate parameter sharing but nothing more—a bag of reusable features, a lookup table of relational patterns. The second conjecture says something stronger about the *mathematical form* of the invariants.

Conjecture 2 (Operator Representation). *The relational invariants of Conjecture 1 can be represented as operators with composability and continuity:*

1. **Composability:** $T(s) \circ T(t) = T(s + t)$. *Relational templates nest: “vindicate” composes judgment + social role + temporal sequence. The operator represents this nesting as algebraic composition.*
2. **Continuity:** $T(\alpha)$ is defined for any real α , not just integers. *Computational depth is continuous, not discrete. The same operator at $\alpha = 2$ gives shallow pattern matching; at $\alpha = 5$ gives deep relational inference.*
3. **Spectral decomposition:** T decomposes into independent modes that can be activated independently. *Different relational patterns (spatial, temporal, causal, social) correspond to different spectral components of the operator. Input-dependent coefficients select which modes to engage.*

These properties are not incidental. They are what makes the invariants *useful* beyond parameter sharing. A lookup table of relational patterns satisfies Conjecture 1 but not Conjecture 2. A bag of reusable features satisfies neither. Only an operator representation provides all three: composability (templates nest algebraically), continuity (depth varies smoothly), spectral decomposition (patterns activate independently).

4.3 The Recovery Conjecture

Given that the invariants exist (Conjecture 1) and have operator form (Conjecture 2), the architectural question becomes:

Conjecture 3 (Operator Recovery). *For an L -layer network with shared coupling $\mathbf{M}$ computing $\mathbf{h}_L = f_L \circ \dots \circ f_1(\mathbf{h}_0)$, there exists a functional T of $\mathbf{M}$ and a nonlinear readout g such that:*

$$\|g(T(\mathbf{M}) \cdot \mathbf{h}_0) - \mathbf{h}_L\| \leq \varepsilon \|\mathbf{h}_L - \mathbf{h}_0\| \quad (4)$$

where ε depends on:

1. **Capacity:** $T(\mathbf{M})$ must have sufficient spectral degrees of freedom. ε decreases as the operator’s spectral capacity increases.
2. **Learnability:** Gradient descent must find T and g jointly. ε depends on the optimisation landscape, not just the function class.

The conjecture says: the state trajectory of an L -layer network can be *recovered* by an operator acting on the initial state, modulo a readout. The recovery is not exact— $\varepsilon > 0$ —but the error is bounded relative to the total orbit distance.

Epsilon is not uniform. ε should not be thought of as a blanket error across all inputs and all tasks. Think of $\mathbf{M}$ ’s eigenspace as a small-world network: a few high-eigenvalue modes are **hubs**—they connect many concepts through short relational paths (subject-verb, cause-effect, agent-patient). These hubs account for most of the predictive value of language, because relational patterns follow a power law.

For anything on the structural backbone—anything reachable through the major relational highways— ε is small. For rare specific associations (“the butler’s name was Jenkins”) that lie off any highway, ε is large. The FFN handles these: it fills the specific slot once the operator has established which slot exists.

ε is inversely weighted by importance: small where it matters (structural backbone, high-frequency patterns), large where it doesn’t (long-tail specifics). The Krylov measurement (36% trajectory recovery) may capture a much larger fraction of the *predictive value*—because the structural part is disproportionately valuable for next-token prediction.

What “recovery” means. We are not asking whether $T(\mathbf{M})\mathbf{h}_0 = \mathbf{h}_L$. We are asking whether $T(\mathbf{M})\mathbf{h}_0$ lands in the *same neighbourhood* of representation space as $\mathbf{h}_L$ —close enough that a nonlinear readout g can extract the same predictions. The quality of recovery should be measured not by trajectory norm but by **downstream prediction accuracy**: does $g(T(\mathbf{M})\mathbf{h}_0)$ produce the same output distribution as the L -layer network? The answer can be “yes” even when the trajectory error is large, if the error lies in dimensions that are orthogonal to the prediction task.

What the FFN does. The readout g is not an afterthought. It is where context meets structure. $T(\mathbf{M})\mathbf{h}_0$ produces a representation that reflects the relational world model: which templates are active, which slots are open, which constraints apply. The FFN interprets this activated landscape: fills the slots with specific tokens, resolves ambiguities, produces the output distribution.

The FFN’s job is made *easier* by the operator having done the structural work first. Predicting “Lily” given the template “a female name goes here” is much easier than predicting “Lily” from raw token statistics. The operator builds the world; the FFN lives in it.

The Lie–Trotter connection. The relationship between the operator-space architecture and the standard ORN is not an analogy—it is a mathematical identity. The Lie–Trotter product formula states:

$$\exp(t(A + B)) = \lim_{n \rightarrow \infty} \left[\exp\left(\frac{tA}{n}\right) \exp\left(\frac{tB}{n}\right) \right]^n \quad (5)$$

In the ORN, $A = \mathbf{M}_{\text{coupling}}$ (shared attention) and $B = \mathbf{M}_{\text{response}}$ (per-layer FFN). Each ORN layer computes $\mathbf{h} \rightarrow \mathbf{h} + \text{Attn}_{\mathbf{M}}(\mathbf{h}) + \text{FFN}(\mathbf{h})$, which approximates $\exp((\mathbf{M}_{\text{coupling}} + \mathbf{M}_{\text{response}})/L)$. Repeating L times gives the Lie–Trotter approximation to the full semigroup.

The error of the n -step Lie–Trotter splitting is $\mathcal{O}([\mathbf{M}_{\text{coupling}}, \mathbf{M}_{\text{response}}] \cdot t^2/n)$, where $[\cdot, \cdot]$ is the **commutator**. If coupling and response commute ($[A, B] = 0$), the splitting is exact at $n = 1$: one coupling step and one response step, in any order, gives the exact answer. As the commutator grows, more splitting steps (more layers) are needed.

The commutator crystallises. We have measured this: the commutator rank $[\mathbf{M}, W_V W_O]$ traces $3 \rightarrow 150 \rightarrow 65$ during training. Coupling and response are **becoming more commutative** as $\mathbf{M}$ crystallises. This predicts: as training progresses (and possibly as scale increases), fewer Lie–Trotter splittings—fewer coupling stages—are needed for the same accuracy.

Our two-stage architecture (couple $\rightarrow$ FFN $\rightarrow$ re-couple $\rightarrow$ FFN) is the $n = 2$ Lie–Trotter splitting. The standard ORN with $L = 8$ is the $n = 8$ splitting. If the commutator continues to shrink during training, the $n = 2$ splitting should converge to the $n = 8$ splitting. This is testable: measure $\|[\mathbf{M}, \text{FFN}]\|$ at different training stages and scales.

Why this matters for the architecture. The Lie–Trotter connection transforms the question “how many coupling stages?” from an empirical guess into a **measurable quantity**—the commutator norm.

- Small commutator $\rightarrow$ few coupling stages needed $\rightarrow$ the operator-space architecture works with 1–2 stages.
- Large commutator $\rightarrow$ many stages needed $\rightarrow$ the architecture converges back toward the ORN (many interleaved coupling-response steps).
- Commutator shrinks with training $\rightarrow$ as $\mathbf{M}$ crystallises, the architecture becomes more operator-like. The crystallisation IS the transition from state-space to operator-space computation.

Why recovery is hard. The conjecture requires that $T(\mathbf{M})$ —a function of a single shared operator—recovers what L layers of input-dependent, nonlinear, sequential computation produce. The L -layer network sees intermediate states: layer 5 knows what layer 4 discovered. The operator composes $\mathbf{M}$ ’s eigenmodes algebraically. Whether the algebraic advantage compensates for the loss of intermediate states depends on the commutator: if coupling and response nearly commute, the intermediate states add little information, and the algebraic composition suffices.

The Krylov measurement (36% recovery) provides a lower bound using the simplest functional (polynomial) on a model where the commutator has not fully converged. Richer functionals and longer training (more commutator reduction) may improve this substantially. The conjecture states that *some* functional achieves small ε on the structurally important components of the trajectory. Finding it is the research programme.

5 Two Tests: Capacity and Learnability

We define two quantities measured on a trained ORN processing input $\mathbf{x}$:

Definition 2 (Orbit distance). The total displacement of the residual stream from input to output:

$$\Delta_{\text{orbit}}(\mathbf{x}) = \|\mathbf{h}_L - \mathbf{h}_0\| \quad (6)$$

This measures how far the operator moves the representation.

Definition 3 (Correction magnitude). The cumulative contribution of perturbative corrections across all layers:

$$\Delta_{\text{corr}}(\mathbf{x}) = \sum_{l=1}^L \|\sigma(g_l(\mathbf{h}_l)) \cdot \text{corr}_l(\mathbf{h}_l)\| \quad (7)$$

This measures how much the nonlinear corrections contribute to the final state.

Conjecture 4 (Operator Dominance). *In an ORN with shared $\mathbf{M}$, trained to convergence on a language modelling task:*

1. *The correction-to-orbit ratio $\rho(\mathbf{x}) = \Delta_{\text{corr}}(\mathbf{x})/\Delta_{\text{orbit}}(\mathbf{x})$ satisfies $\rho < \epsilon$ for some small ϵ (e.g., $\epsilon < 0.15$) across the input distribution.*
2. *ρ decreases during training as $\mathbf{M}$ crystallises: the operator converges to a good approximation and corrections become smaller relative to the orbit.*
3. *ρ decreases with model scale: larger models learn better operators that require less perturbative correction.*

If Conjecture 4 holds, it implies that the “knowledge” of the ORN is primarily encoded in the operator $\mathbf{M}$ and its semigroup structure, not in the per-layer nonlinear transformations. The corrections are context-dependent residuals: they handle what the operator cannot, but the operator handles most of what matters.

The path integral analogy. In quantum field theory, the partition function decomposes as:

$$Z = \underbrace{\int \mathcal{D}\phi e^{iS_0[\phi]}}_{\text{free theory (Gaussian)}} \cdot \underbrace{\left(1 + \sum_n \lambda^n V_n[\phi]\right)}_{\text{interactions (perturbative)}} \quad (8)$$

The free theory S_0 is quadratic and exactly solvable. The interaction terms V_n are perturbative corrections controlled by a coupling constant λ . Perturbation theory works when λ is small—when the interactions are genuinely perturbative.

In the ORN:

- The **free theory** is $T(\mathbf{h}_0) = \exp(L \cdot \mathbf{M})\mathbf{h}_0$ —the linear semigroup propagation.
- The **interactions** are the per-layer FFN and gated corrections.
- The **coupling constant** is the correction-to-orbit ratio ρ .

Conjecture 4 states that ρ is small: the ORN operates in the perturbative regime. This is not guaranteed—if $\rho \approx 1$, the corrections are doing as much work as the operator, and the decomposition is meaningless.

6 What “Learning in Operator Space” Means

The conjecture is not merely about approximation. It is about what the network *learns*—and whether an entirely new kind of architecture is justified.

The current ORN is a state-space machine that happens to share an operator. It does not compose operators, defer materialisation, or exploit semigroup structure computationally. But if the conjecture holds—if ρ is small and the linear operator does the heavy lifting—then the state-space computation is *redundant overhead*. The L sequential layers are laboriously materialising intermediate states that are largely determined by a single linear operator $\mathbf{M}$. An architecture that computes $T = f(\mathbf{M})$ directly and materialises state once would produce the same outputs with less computation and richer algebraic structure.

Why this matters. The difference between state-space and operator-space is not optimisation. It is a difference in *what the architecture can express and learn*.

In state space, the network learns a sequence of transformations. Knowledge is distributed across layer-specific parameters and the intermediate states they produce. There is no notion of “composing” knowledge, “transferring” a relational structure, or “varying the depth” continuously.

In operator space, the network learns a *single relational structure* $\mathbf{M}$ whose algebraic properties (spectrum, rank, semigroup composition) are first-class objects. Three capabilities follow that state-space architectures do not have:

1. Transfer by design. If knowledge is in the operator, it transfers: a new task needs new response heads but the same $\mathbf{M}$. The current ORN provides indirect evidence: frozen- $\mathbf{M}$ transfer shows only +0.73% gap [38]. But this is tested within a state-space machine. An operator-space architecture would make transfer a *structural property*, not an empirical observation.

2. Algebraic composition. Operators compose; states do not. $T(s) \circ T(t) = T(s + t)$ is a property of the semigroup. An operator-space architecture inherits this: two operators trained on different tasks could be composed algebraically, without retraining. No state-space architecture can do this because there is no algebraic structure on intermediate states.

3. Continuous depth. In state space, depth is an integer L chosen at design time. In operator space, depth is a continuous parameter: the exponent α in $\exp(\alpha\mathbf{M})$, the degree K in a polynomial $p(\mathbf{M})$, or the convergence threshold of a fixed-point iteration. This makes adaptive computation natural, not bolted on.

7 Experimental Programme

We propose ten experiments organised into three groups.

7.1 Group I: Is the Operator Dominant? (Experiments 1–5)

These experiments measure whether the operator $\mathbf{M}$ does the heavy lifting in a trained ORN, or whether the corrections are load-bearing.

Experiment 1: Orbit Distance vs Correction Magnitude. On the V2 checkpoint (108M, 8B tokens), measure Δ_{orbit} and Δ_{corr} across 1000 random inputs. Compute $\rho = \Delta_{\text{corr}}/\Delta_{\text{orbit}}$ per input.

- **Dominance holds if:** median $\rho < 0.15$ (corrections are $< 15\%$ of orbit distance).
- **Dominance fails if:** median $\rho > 0.5$ (corrections are comparable to orbit).
- **Also measure:** per-layer breakdown. Do corrections grow with depth? Do they correlate with $\mathbf{M}$ ’s attention entropy?

Experiment 2: Correction-to-Orbit Ratio During Training. Train a small ORN (20M) from scratch. At each checkpoint, measure ρ on held-out data.

- **Conjecture predicts:** ρ decreases as training progresses, with a phase transition near $\mathbf{M}$ ’s crystallisation point.
- **Falsified if:** ρ is flat or increases during training.
- **Also track:** $\mathbf{M}$ ’s effective rank, condition number, and ρ simultaneously. If they co-vary, crystallisation IS operator convergence.

Experiment 3: The Ablation Spectrum. After training, evaluate five models:

- (a) Full ORN (baseline).
- (b) Corrections zeroed out (set gate bias to $-\infty$).
- (c) FFN zeroed out.
- (d) Both corrections and FFN zeroed out (pure $\mathbf{M}$ orbit).
- (e) $\mathbf{M}$ replaced with random orthogonal matrix (corrections and FFN intact).
 - If (d) retains $> 50\%$ of (a)’s quality: $\mathbf{M}$ alone carries most of the information.
 - If (e) collapses to chance: $\mathbf{M}$ ’s learned structure is essential, not just any operator.
 - The gap between (a) and (b) quantifies exactly what corrections contribute.

Experiment 4: Recovery Test. After training, freeze $\mathbf{M}$ and reset all correction weights to random. Retrain only corrections (1% of parameters) for 10% of the original training steps.

- If quality recovers to within 5% of the original: the operator truly contains the knowledge, and corrections are learnable polish.
- If quality does not recover: corrections encode information that is not recoverable from $\mathbf{M}$ alone. The operator is incomplete.

Experiment 5: Correction Scaling Law. Train ORNs at 5 scales (2M, 10M, 50M, 108M, and the 605M V3 if available). Measure ρ at each scale after convergence.

- **Conjecture predicts:** ρ decreases with scale (larger operators need less correction).
- **Plot:** ρ vs d on log-log axes. If linear, extract the scaling exponent.
- **The dream result:** $\rho \propto d^{-\alpha}$ for some $\alpha > 0$, implying corrections vanish at scale.

7.2 Group II: Can We Compute Directly in Operator Space? (Experiments 6–10)

These experiments test whether practical architectures can compute in operator space—applying $\mathbf{M}$ ’s semigroup structure without materialising intermediate states—while matching state-space quality.

Experiment 6: Matrix Exponential ORN. Replace L layers with $\mathbf{h} = \exp(\alpha\mathbf{M}) \cdot \mathbf{h}_0$ plus a single nonlinear readout.

- Tests: does the free propagator suffice?
- The learned scalar α controls “depth” continuously.
- Precedent: Fischbacher & Comsa (2020) showed a single expm layer with 150K params outperforms 8M-param FC nets [7].

Experiment 7: Spectral Propagator ORN. Decompose $\mathbf{M} = U\Sigma V^\top$. Learn per-eigenmode scaling functions $f(\sigma_i)$ instead of uniform $\exp(\alpha\sigma_i)$.

- Tests: is $\mathbf{M}$ ’s spectrum the right computational primitive?
- Precedent: Fourier Neural Operators [8] truncate high-frequency modes, analogous to $\mathbf{M}$ ’s rank crystallisation.

Experiment 8: Polynomial Operator ORN. Compute $\mathbf{h} = \sum_{k=0}^K c_k \mathbf{M}^k \mathbf{h}_0$ with learned coefficients c_k .

- Tests: can learned polynomials of $\mathbf{M}$ replace sequential layers?
- Degree K controls “depth” discretely. Cayley–Hamilton guarantees $K \leq d$ suffices.
- Precedent: ChebNet [9] and CayleyNet [10] use polynomials/rationals of graph Laplacians.

Experiment 9: S4-Style Depth Convolution. Use $\mathbf{M}$ as the state matrix in a linear recurrence over depth, converted to parallel convolution.

- Tests: can depth-wise operator composition be parallelised?
- Precedent: S4 [1] converts sequential recurrence to parallel convolution via FFT.
- $\mathbf{M}$ ’s role is analogous to S4’s HiPPO matrix, but learned rather than prescribed.

Experiment 10: Deferred Operator Composition. Each virtual layer produces a low-rank update $T_l = I + \alpha_l u_l v_l^\top$. Updates are composed multiplicatively before application: $\mathbf{h} = T_L \cdots T_1 \cdot \mathbf{h}_0$.

- Tests: true deferred materialisation with learned composition.
- No intermediate states exist until the composed operator is applied.
- Precedent: multiplicative interactions are strictly more expressive than additive [11].

7.3 Kill Criteria

Experiment	Kill if	Interpretation
1 (orbit vs correction)	median $\rho > 0.5$	Corrections are not perturbative
2 (ratio during training)	ρ flat or increasing	Crystallisation doesn't help
3 (ablation spectrum)	Pure $\mathbf{M}$ orbit $< 20\%$ of full	$\mathbf{M}$ is not the primary carrier
5 (scaling law)	ρ increases with scale	Operator dominance doesn't scale
6–10 (architectures)	All $> 30\%$ worse than baseline	Operator-space computation doesn't work

8 Literature Review: Support and Challenges for the Conjectures

We survey results across AI, mathematics, and physics that bear on the three conjectures. The literature is organised by what it supports or challenges, not by field.

8.1 Strong Support for Structural Invariance (Conjecture 1)

Weight sharing improves compositional generalisation. Csordas et al. [15] show that Universal Transformer variants (which share all weights across layers) dramatically improve compositional generalisation on COGS and SCAN, from 35% to 81%. The paper demonstrates that weight sharing broadly helps composition, supporting Conjecture 1: relational invariants exist and sharing them improves generalisation. However, Csordas et al. share *all* weights (like ALBERT); the specific ORN decomposition—share only coupling (A, B) , keep response (W_V, W_O, FFN) per-layer—has not been independently discovered. The ORN's contribution is identifying *which* parameters encode the invariant, based on spectral invariance measurements.

The Platonic Representation Hypothesis. Huh et al. [16] (ICML 2024) show that different neural networks converge to similar representations at scale. Their metric (mutual k -nearest-neighbour alignment) reaches 0.16 out of 1.0 cross-modally, which they acknowledge is modest: “lots left to explain.” They measure representation alignment at the *kernel* level (distance structures between embeddings) but do not analyse attention patterns, spectral structure, or coupling geometry.

Our spectral invariance measurement operates at a deeper level: the coupling *operator's* spectrum, not the embedding distances. The ~ 0.98 correlation across GPT-2, LLaMA, Mistral, Phi, and OLMo is stronger evidence for the Platonic hypothesis than anything in their paper—it shows convergence at the level of the relational operator, not just the induced kernel. Our frozen- $\mathbf{M}$ transfer experiment (+0.73% gap) directly tests what they hypothesise but do not measure: that the coupling geometry transfers because it reflects a data invariant, not an architectural artefact.

RRT validates low-rank corrections. Relaxed Recursive Transformers [17] (ICLR 2025) share all weights across layers and recover 99.7% of quality at 1.4B–2.6B scale using per-layer LoRA corrections of rank 32–128. The per-layer variation is provably low-rank: a small perturbation of a shared structure.

Geometric deep learning framework. Bronstein et al. [18] unify successful deep learning architectures as enforcing geometric priors (symmetries). The ORN adds a new symmetry class: depth-invariance of coupling. The framework predicts that architectures exploiting this symmetry will be more parameter-efficient—confirmed by the $48\times$ compression.

MoE expert specialisation. Muennighoff et al. [19] analyse expert specialisation in mixture-of-experts models and find that experts specialise by *syntax* (subjects, predicates, modifiers) more than semantics, and this specialisation is consistent across layers. This is independent evidence that syntactic coupling is an invariant—the same claim as Conjecture 1.

8.2 Strong Support for Operator Representation (Conjecture 2)

Koopman operator theory. The Koopman framework [20] proves that every nonlinear dynamical system has a linear (but infinite-dimensional) operator representation with composability ($K^{s+t} = K^s K^t$), continuity, and spectral decomposition—exactly the properties of Conjecture 2. Lusch et al. [21] show that autoencoders can learn finite-dimensional Koopman approximations whose eigenmodes correspond to interpretable physical modes (oscillation frequencies, decay rates). Their training loss enforces three constraints: reconstruction ($\|\mathbf{x} - \phi^{-1}(\phi(\mathbf{x}))\|$), linearity ($\|\phi(\mathbf{x}_{k+1}) - K\phi(\mathbf{x}_k)\|$), and prediction ($\|\mathbf{x}_{k+1} - \phi^{-1}(K\phi(\mathbf{x}_k))\|$).

A critical difference: in Lusch et al., the spectral structure of K is *prescribed* (block-diagonal with (μ, ω) parameterisation enforcing circular eigenfunction symmetry). In the ORN, $\mathbf{M}$ ’s spectral structure *emerges*: rank drops from 189 to ~ 70 , condition number rises, spectral shape locks by 3B tokens. The emergence is stronger evidence for Conjecture 1 than prescribed structure—it means gradient descent discovers the spectral decomposition, not the architect.

Koopman closure is provably impossible for language. Colbrook et al. [22] prove that finite-dimensional Koopman learning fails for generic nonlinear systems. More precisely: any system with multiple fixed points (attractors) cannot have a finite-dimensional Koopman-invariant subspace containing the state observables, because topological conjugacy preserves the number of fixed points and a finite-dimensional linear system has exactly one. Language has effectively infinite attractors (every sentence is a trajectory toward a different fixed point). Therefore, exact finite-dimensional Koopman closure for language is **provably impossible**.

This is not a weakness of the programme—it is why the perturbative corrections exist. The corrections break the Koopman linearity at each layer, compensating for the closure residual that finite-rank $\mathbf{M}$ cannot absorb. Koopman theory says $\mathbf{M}$ captures the dynamics within its invariant subspace (the 70-dimensional coupling geometry); the corrections handle everything outside it. The rank-70 $\mathbf{M}$ is the best finite-dimensional Koopman approximation to the coupling dynamics of language. The corrections are the theoretically necessary residual.

The synthesis. Combining the Platonic and Koopman perspectives: the Platonic hypothesis predicts that a universal coupling kernel exists (all models converge to it). Koopman theory predicts that this kernel has a finite-rank operator approximation with spectral decomposition—but the approximation is inexact, with a residual controlled by the closure gap. $\mathbf{M}$ is the finite-rank Koopman approximation to the Platonic kernel. The corrections handle the closure residual. Both theories predict the same object from different directions; neither alone is sufficient.

Semigroup spectral theory. Engel & Nagel [23] prove the spectral mapping theorem for semigroups: if $\mathbf{M}$ has eigenvalues $\{\lambda_i\}$, then $T(t) = \exp(t\mathbf{M})$ has eigenvalues $\{\exp(t\lambda_i)\}$. Each eigenmode evolves independently—property (c) of Conjecture 2 holds automatically for matrix-generated semigroups.

The Lie–Trotter product formula. The ORN’s alternating structure (shared attention then per-layer FFN) is the Lie–Trotter decomposition [24] of $\exp(t(\mathbf{M}_{\text{coupling}} + \mathbf{M}_{\text{response}}))$ into alternating applications of the coupling and response semigroups. This is not an approximation discovered by gradient descent—it is the standard numerical method for operator splitting, built into the architecture.

DAT confirms asymmetry. Altabaa & Lafferty [?] (ICML 2025) independently decompose attention into tensor products and prove that asymmetric factorisation ($A \neq B$) is essential for autoregressive tasks. This confirms our finding that symmetric $LL^\top$ fails while $AB^\top$ succeeds.

8.3 Strong Support for Recovery (Conjecture 3)

Universality of linear recurrence + nonlinear readout. Orvieto et al. [6] prove that a diagonal linear RNN followed by a pointwise MLP is a universal approximator for causal sequence-to-sequence maps. The linear recurrence is our $T(\mathbf{M})$; the MLP is our readout g . This is Conjecture 3 proven for the state-space model case.

Chebyshev approximation bounds. For a matrix $\mathbf{M}$ with condition number κ , a polynomial of degree k approximates any analytic function of $\mathbf{M}$ with error $\mathcal{O}((\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1})^k)$ [25]. For $\kappa \approx 10$ (our $\mathbf{M}$ ’s condition number), degree 20 gives error $< 10^{-6}$. At $L = 8$: error ≈ 0.004 , consistent with the +0.73% frozen- $\mathbf{M}$ transfer gap.

ChebNet and CayleyNet. Defferrard et al. [9] and Levie et al. [10] prove that polynomial and rational filters of a shared operator (the graph Laplacian) are universal for graph signal processing. Our polynomial functional $T(\mathbf{M})$ is structurally identical—a ChebNet filter applied in the depth dimension rather than the graph dimension.

MERA and tensor networks. Vidal’s MERA [26] decomposes quantum states into hierarchical operators: shared disentanglers (coupling) and per-scale isometries (response). Levine et al. [27] prove that weight-tied deep networks have MERA-like tensor network structure, and that weight sharing *increases* expressiveness per parameter (via entanglement entropy). The ORN is structurally a 1D MERA where $\mathbf{M}$ is the disentangler and the response heads are isometries.

8.4 Results That Challenge the Conjectures

Linear attention impossibility. Ali et al. [28] and Arora et al. [29] prove that linear operators cannot solve associative recall tasks that softmax attention solves in $\mathcal{O}(1)$ layers. $T(\mathbf{M})$ faces the same bottleneck: it stores at most d^2 bits of key-value associations. The nonlinear readout g (FFN + corrections) is essential, not optional. The ε in Conjecture 3 cannot be made arbitrarily small by $T(\mathbf{M})$ alone.

Chomsky hierarchy limitations. Deletang et al. [30] prove that shared-weight transformers without adaptive mechanisms are limited to TC^0 (regular languages with counting). The perturbative corrections are necessary for expressiveness beyond this class. Conjecture 3 holds only with corrections included in g .

Our own Krylov measurement. 36% recovery, 92% residual. The polynomial of $\mathbf{M}$ captures only the coupling component—the structural fraction—not the full computation. Conjecture 3’s ε is bounded below by the FFN’s structural contribution ($\sim 64\%$ of the trajectory). The conjecture should be understood as: $T(\mathbf{M})$ captures the *relational structure*, not the full state.

8.5 Novel Connections

Several results suggest structural parallels that have not been previously noted:

- **Renormalisation group:** $\mathbf{M}$ ’s crystallisation (liberation $\rightarrow$ crystallisation dynamics, rank $534 \rightarrow 69$) IS an RG flow. The spectral modes of $\mathbf{M}$ decompose into relevant (survive

across depth) and irrelevant (wash out). Koch-Janusz & Ringel [31] prove that RG maximises mutual information between the coarse description and long-range structure— $\mathbf{M}$ ’s crystallisation may be optimal in this information-theoretic sense.

- **Gauge theory:** $\mathbf{M}$ is a gauge connection on a fibre bundle where the base space is the token sequence and the fibres are representation spaces. Spectral invariance across layers IS gauge invariance. Van Nierop [32] formalises this mapping; Bronstein et al. [18] provide the geometric deep learning framework.
- **Transfer matrix:** $\mathbf{M}$ is a transfer matrix for the residual stream. Baxter’s exactly solved models [33] show that all thermodynamic quantities (= all output statistics) are determined by the transfer matrix’s eigenvalues. DMRG truncation [34] to top- k eigenvalues with controlled error is exactly $\mathbf{M}$ ’s rank crystallisation.
- **Effective field theory:** Roberts et al. [35] prove that deep networks admit a perturbative expansion around the infinite-width (linear) limit. $T(\mathbf{M})$ is this leading term; the FFN readout captures the nonlinear corrections. The perturbative gate (anticorrelating with entropy at $r = -0.95$) controls the expansion parameter.

9 The Orbital Network: Depth Separated from Width

Every existing architecture entangles depth and width. Each layer performs both token-token interaction (attention, convolution, recurrence) and per-token transformation (FFN, projection, nonlinearity), repeated L times. Token 5 at layer 1 sees token 3’s raw embedding; token 5 at layer 8 sees token 3’s representation after 7 layers of transformation. The two operations are inseparable—you cannot have deeper representations without re-routing between tokens at every stage.

The Orbital Network untangles them.

Depth is algebraic. A shared operator $\mathbf{M}$ is composed with itself as a polynomial $p(\mathbf{M}) = \sum_{k=0}^K c_k \mathbf{M}^k$ and applied to each token embedding *independently*. No token-token communication during the depth pass. The “depth” is the polynomial degree K , not a layer count. $\mathbf{M}^8$ means “8 layers deep” but computed as one algebraic evaluation.

Width is attention. A single causal attention layer handles all token-token interaction. This is where causality is enforced, where context is read, where tokens influence each other. It happens once, not L times.

Readout is nonlinear. A single FFN plus projection head. This is where all nonlinearity lives. The depth pass is entirely linear (polynomial of a matrix); the attention is softmax-nonlinear but shallow; the FFN provides the expressivity that the linear depth cannot.

The forward pass:

1. Embed: $\mathbf{h}_0 = \text{tok_emb}(\mathbf{x}) + \text{pos_emb}$.
2. Depth: $\mathbf{h} = p(\mathbf{M})\mathbf{h}_0 = \sum_{k=0}^K w_k \odot (\mathbf{M}^k \mathbf{h}_0)$, where $w_k \in \mathbb{R}^d$ are learned per-dimension weights for degree k . Each token is transformed independently.
3. Width: $\mathbf{h} \leftarrow \mathbf{h} + \text{CausalAttn}(\mathbf{h})$.
4. Readout: $\mathbf{h} \leftarrow \mathbf{h} + \text{FFN}(\mathbf{h})$, then logits = head(LN($\mathbf{h}$)).

This is not a transformer with shared weights. It is not a state-space model. It is not a DEQ. The depth pass has no recurrence, no fixed-point iteration, no intermediate states. It is a closed-form algebraic evaluation.

This is deliberately extreme. The purpose of flattening—collapsing L layers into one polynomial plus one attention layer—is diagnostic, not architectural. We flatten to learn the decomposition: which component carries coupling, which carries response, where the bottleneck is,

what breaks. The flattening experiments teach us the decomposition. The scaling experiments will teach us how to re-stack.

The final architecture will almost certainly re-introduce depth: multiple coupling stages with shared $\mathbf{M}$, each richer than a standard attention layer (polynomial depth, input-dependent coefficients, asymmetric factored coupling), separated by nonlinear response blocks. The number of coupling stages should scale with task complexity: two suffice for TinyStories; centre-embedded syntax or multi-step reasoning may need four or five. The insight from flattening is not “use two stages forever.” It is: **coupling is the scarce resource, response is the commodity**, and now we know how to build each one.

But first: does the extreme version work at all? It still has a separate attention layer for token-token interaction, using $\mathbf{M}$ for coupling. This may be one step too conservative.

10 Geometric Attention: Coupling as a Field, Not a Matrix

Standard attention computes an $S \times S$ matrix of pairwise couplings:

$$\text{Attn} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right), \quad Q = \mathbf{h}W_Q, \quad K = \mathbf{h}W_K \quad (9)$$

This matrix explicitly represents every relationship between every pair of tokens. It is materialised, stored, and applied. It is a lookup table of couplings.

But attention is not a table. When you read a sentence, your visual system does not compute a matrix of pairwise relevances over every word on the page. Your reading is guided by a trained dynamical system—saccade patterns shaped by years of experience—that moves through the text, slowing on unfamiliar constructions, skipping predictable ones. The “attention” to the next word is not computed from a coupling matrix. It emerges from the reading system’s trained geometry interacting with the current input. It changes with every fixation, but the underlying system does not change—only the state moving through it.

This is the difference between Newtonian gravity (compute an $N \times N$ force matrix between all bodies) and general relativity (define a metric and let bodies follow geodesics). In GR, there is no force matrix. The coupling between bodies is implicit in the curvature of spacetime. Two masses “attend” to each other because they curve the same region of space.

10.1 The Proposal: Attention from Geometry

If $p(\mathbf{M})$ warps the embedding space, then the inner products in the warped space implicitly define a coupling pattern:

$$\mathbf{h}_{\text{warped}} = p(\mathbf{M}) \cdot \mathbf{h}_0 \quad (10)$$

$$C_{ij} = \mathbf{h}_{\text{warped},i}^\top \mathbf{h}_{\text{warped},j} \quad (11)$$

No W_Q , no W_K . The coupling matrix C is never explicitly parameterised—it emerges from the geometry of the warped space. Tokens that $\mathbf{M}$ ’s polynomial brings close together (high inner product) couple strongly. Tokens that remain far apart couple weakly.

The architecture becomes:

1. Embed: $\mathbf{h}_0 = \text{tok_emb}(\mathbf{x}) + \text{pos_emb}$.
2. Warp: $\mathbf{h}_w = p(\mathbf{M}) \cdot \mathbf{h}_0$ (polynomial of $\mathbf{M}$, as before).
3. Couple: $C = \text{causal_mask}\left(\mathbf{h}_w \mathbf{h}_w^\top / \sqrt{d}\right)$ (inner products in warped space).
4. Respond: $\mathbf{v} = W_V \mathbf{h}_w$, then $\mathbf{h} = \text{softmax}(C) \cdot \mathbf{v}$ (gather values by geometric proximity).
5. Readout: FFN + head.

There is no attention layer. The operator IS the attention, expressed as a geometric field rather than a pairwise matrix. The $S \times S$ coupling pattern emerges from $d \times d$ operator geometry.

10.2 Why This Is Different from Attention with Shared Q/K

The Orbital Network (Section 9) uses $\mathbf{M}$ for Q and K: $q = \mathbf{h} @ A$, $k = \mathbf{h} @ B$. This is still explicit attention—it computes $QK^\top$, materialises the coupling matrix, applies softmax. $\mathbf{M}$ provides the projection but the mechanism is unchanged.

Geometric attention is structurally different. The coupling is not $(\mathbf{h}A)(\mathbf{h}B)^\top = \mathbf{hMh}^\top$ (a bilinear form applied to the *current* state). It is $p(\mathbf{M})\mathbf{h}_0 \cdot (p(\mathbf{M})\mathbf{h}_0)^\top$ (inner products of *operator-transformed* embeddings). The polynomial warp means different tokens are transformed by different effective operators—because $\mathbf{M}^k \mathbf{h}_{0,i}$ projects token i onto $\mathbf{M}$ ’s eigenmodes with position-dependent coefficients.

The coupling at token 50 differs from token 5 not because the coupling rule changed but because the accumulated context (earlier tokens in the causal mask) projects differently onto $\mathbf{M}$ ’s eigenvectors. **The field is invariant; the state moving through it evolves.**

10.3 The Polynomial Degree as Field Resolution

The degree K of the polynomial controls the complexity of the coupling field:

- $K = 1$: $\mathbf{h}_w = c_0 \mathbf{h}_0 + c_1 \mathbf{M} \mathbf{h}_0$. The field is nearly flat—coupling is determined by raw embedding similarity plus a single linear warp. This captures bigram-level patterns.
- $K = 3$: the field has curvature. Tokens couple through three levels of $\mathbf{M}$ ’s spectral structure—enough for syntactic patterns (subject-verb agreement, modifier binding).
- $K = 8$: the field has rich topology. Long-range dependencies, nested constructions, narrative arcs.

The degree is not “depth” in the transformer sense (number of sequential processing stages). It is the **resolution of the coupling field**—how many of $\mathbf{M}$ ’s eigenmodes contribute to the geometry through which tokens interact.

10.4 Capacity and Learnability

The same two challenges apply.

Capacity. Can geometric attention match explicit softmax attention? Softmax can produce arbitrarily sharp coupling (one token attends to exactly one other). Inner products in warped space produce smoother coupling—proximity is gradual, not binary. Can the polynomial warp produce sharp enough geometry?

The polynomial has $K \times d$ parameters controlling the warp. Softmax attention has $2d^2$ parameters (W_Q, W_K). For $K \ll d$, the polynomial has fewer parameters—but it operates on the full spectral structure of $\mathbf{M}$, which may compress the coupling more efficiently than explicit projections.

If $\mathbf{M}$ has effective rank r , the coupling field lives in an r -dimensional subspace. The $S \times S$ coupling matrix has rank at most r —which is a strong structural prior. Explicit attention has no such constraint (it can produce full-rank $S \times S$ matrices). Whether rank- r coupling suffices for language modelling is an empirical question.

Learnability. $\mathbf{M}$ must learn a geometry that produces useful coupling patterns through inner products alone, without the help of separate Q/K projections. This is a harder optimisation target than learning Q/K matrices, because the coupling is an indirect consequence of the warp rather than a directly parameterised quantity. The gradients flow through $p(\mathbf{M})$, through

the inner product, through softmax, through the value gather—a longer path than standard attention.

The degree curriculum and two-timescale learning apply here as well. Low-degree coupling (nearly linear, easy to learn) first; high-degree coupling (complex field topology) after $\mathbf{M}$ has crystallised.

10.5 Asymmetric Geometric Attention

Equations 10–11 have a fatal flaw: inner products are symmetric. $C_{ij} = \mathbf{h}_{w,i}^\top \mathbf{h}_{w,j} = C_{ji}$. But language coupling is directed: “the” attends to “cat” differently than “cat” attends to “the.” Relationships are directed—“vindicate” implies an arbiter judging a claimant, not vice versa.

The fix comes from $\mathbf{M}$ ’s own structure. $\mathbf{M} = AB^\top$ is already asymmetric (asymmetry ratio 1.41 in V2; symmetric $LL^\top$ fails on autoregressive tasks). Every power of $\mathbf{M}$ factors through A and B :

$$\mathbf{M}^k = A(B^\top A)^{k-1}B^\top \quad (12)$$

So any polynomial of $\mathbf{M}$ factors as:

$$p(\mathbf{M}) = A \cdot q(G) \cdot B^\top, \quad G = B^\top A \quad (13)$$

where $G \in \mathbb{R}^{r \times r}$ is the **inner geometry**—the coupling structure in the reduced space defined by A and B .

The coupling between tokens becomes:

$$C_{ij} = \mathbf{h}_i^\top p(\mathbf{M}) \mathbf{h}_j = \underbrace{(A^\top \mathbf{h}_i)^\top}_{\text{query}_i} \cdot \underbrace{q(G)}_{\text{depth}} \cdot \underbrace{B^\top \mathbf{h}_j}_{\text{key}_j} \quad (14)$$

This is **inherently asymmetric**: $A \neq B$, so query $\neq$ key. Token i projects through A (“how I ask”), token j projects through B (“what I am”). The polynomial $q(G)$ composes relational templates *between* them in the reduced r -dimensional space.

Efficiency. If $A, B \in \mathbb{R}^{d \times r}$, then $G \in \mathbb{R}^{r \times r}$. The polynomial operates in r -dimensional space, not d -dimensional. For $r = 64$, $d = 576$: the polynomial is 64×64 instead of 576×576 — $80\times$ cheaper. The query and key projections are standard matrix multiplications.

Input-dependent depth on the query side.

$$C_{ij} = (A^\top \mathbf{h}_i)^\top \cdot p_i(G) \cdot (B^\top \mathbf{h}_j) \quad (15)$$

where p_i depends on $\mathbf{h}_i$ via a coefficient network. Each token *as a query* selects how deeply to probe the relational geometry:

- “Vindicate” at position i : high-degree polynomial, activates the three-party template.
- “The” at position i : degree 0–1, basic adjacency.

The key side ($B^\top \mathbf{h}_j$) is passive—it presents what the token is. The query side decides how deeply to engage with G ’s spectral structure.

The ORN decomposition, completed.

Component	Role
A	“How I ask” (query projection, $d \rightarrow r$)
B	“What I am” (key projection, $d \rightarrow r$)
$G = B^\top A$	Relational grammar (inner geometry, $r \times r$)
$q_i(G)$	“How deeply to apply the grammar” (input-dependent polynomial)
W_V, W_O	“What to do with the coupling” (response)

$\mathbf{M}$'s asymmetry IS the query-key asymmetry. The factorisation provides both without separate W_Q, W_K matrices. The polynomial provides depth without separate layers. The coefficient network provides input-dependence without per-layer parameters.

The experiment. Replace the symmetric geometric attention in the current experiments with asymmetric geometric attention. Three comparisons: (a) symmetric inner-product coupling (the current geometric attention); (b) asymmetric $q^\top q_i(G)k$ coupling (this section); (c) explicit M-coupled attention ($q = \mathbf{h}A, k = \mathbf{h}B$, standard softmax) as control. If (b) > (a): asymmetry is the missing ingredient. If (b) $\approx$ (c): the polynomial depth in G -space doesn't add beyond what explicit attention provides. If (b) > (c): asymmetric geometric attention with polynomial depth outperforms explicit attention—the geometric field carries real capacity.

10.6 The Capacity Problem: Global Polynomials Are Linear Maps

A polynomial $p(\mathbf{M}) = \sum c_k \mathbf{M}^k$ with global (input-independent) coefficients is a $d \times d$ matrix. It does not matter if $K = 1$ or $K = 100$: the result is $W\mathbf{h}_0$ for some fixed $W = p(\mathbf{M})$. One matrix multiplication. Capacity: d^2 parameters. Same as a single linear layer.

A global polynomial carries zero additional capacity beyond a linear projection.

The Krylov measurement confirmed this: the polynomial subspace recovers only 36% of the orbit. The remaining 64% comes from nonlinear, input-dependent, compositional computation that no fixed polynomial can represent.

A transformer with L layers computes $f_L \circ \dots \circ f_1(\mathbf{h}_0)$ where each f_l is a *different function conditioned on the current state*. The effective function changes with every input. The capacity grows with depth because each nonlinear layer routes information based on what previous layers discovered. A fixed polynomial cannot match this.

10.7 Input-Dependent Polynomials: Where Capacity Lives

The operator carries genuine computational capacity only if the polynomial coefficients are a function of the input:

$$c_k^{(i)} = g(\mathbf{h}_{0,i}), \quad T_i = \sum_k c_k^{(i)} \mathbf{M}^k \quad (16)$$

where g is a small network (the “coefficient network”) and i indexes token positions. Now each position gets a *different* effective operator T_i , which is a different polynomial of the *same* $\mathbf{M}$.

The coupling in geometric attention becomes:

$$C_{ij} = (\mathbf{h}_{0,i})^\top p_i(\mathbf{M})^\top p_j(\mathbf{M}) \mathbf{h}_{0,j} \quad (17)$$

This is a product of two input-dependent polynomials of $\mathbf{M}$ —a degree- $2K$ polynomial with position-dependent coefficients. This IS richer than standard attention ($\mathbf{h}^\top \mathbf{M} \mathbf{h}$, one bilinear form). Different tokens engage different eigenmodes of $\mathbf{M}$ at different intensities. The coupling depends on *how each token projects through its own spectral filter*. This is genuine computation with capacity proportional to the coefficient network.

10.8 The Harder Constraint: Re-Coupling After Transformation

Even with input-dependent coefficients, the polynomial applies to $\mathbf{h}_0$ —it never sees intermediate states. In a transformer, layer 3 sees the result of layers 1–2 including their FFN transformations. The attention at layer 3 is conditioned on what layers 1–2 discovered. The polynomial $p(\mathbf{M})\mathbf{h}_0$ bypasses this: it transforms $\mathbf{h}_0$ directly, regardless of what any FFN has done.

The Krylov data is precise about where this matters. Layers 1–4 (96% linear): $\mathbf{M}$ alone handles the coupling. No intermediate FFN output is needed. Layers 5–8 (50%+ nonlinear):

the FFN’s output *must* inform the next coupling. The coupling at layer 6 depends on the FFN transformation at layer 5.

This means: **the architecture needs re-coupling after transformation.** Not once. Not eight times. But at the transition point—around layers 4–5, where nonlinearity becomes load-bearing.

10.9 The Theoretically Honest Architecture

Two coupling stages, four FFN stages. $\mathbf{M}$ is shared between both couplings. The polynomial coefficients are different—stage 1 uses $c_k(\mathbf{h}_0)$, stage 2 uses $c'_k(\mathbf{h})$ where $\mathbf{h}$ includes the FFN-transformed state.

1. **Input-dependent warp:** $c_k = g(\mathbf{h}_0)$; $\mathbf{h}_w = \sum c_k \odot \mathbf{M}^k \mathbf{h}_0$. Each token selects its spectral depth.
2. **Geometric coupling:** $C = \mathbf{h}_w \mathbf{h}_w^\top / \sqrt{d}$ (causal-masked); value gather via W_V, W_O .
3. **Nonlinear response:** FFN₁, FFN₂. Transform the coupled representation.
4. **Re-couple:** $c'_k = g'(\mathbf{h})$; $\mathbf{h}'_w = \sum c'_k \odot \mathbf{M}^k \mathbf{h}$. Re-warp using the *transformed* state—the second coupling sees what the FFN discovered.
5. **Nonlinear response:** FFN₃, FFN₄. Final transformation.
6. **Readout:** head(LN($\mathbf{h}$)).

How this differs from a transformer. A transformer: 8 separate Q/K/V/FFN stacks, each with independent coupling parameters. This: 2 coupling stages with shared $\mathbf{M}$, input-dependent polynomial coefficients, 4 FFN stages. The coupling geometry is learned once and shared. The routing (polynomial coefficients) is input-dependent and stage-dependent. The second coupling sees the FFN-transformed state, providing the compositional depth that a single polynomial lacks.

Capacity accounting.

- Polynomial coupling: $K \times d \times h$ parameters per stage (coefficient network) + $\mathbf{M}$ ’s d^2 parameters (shared).
- Each FFN: $d \times 4d \times 2$ parameters.
- Total: $2 \times \text{coeff_net} + 4 \times \text{FFN} + d^2$. Fewer parameters than $8 \times (\text{attention} + \text{FFN})$, but algebraically structured through shared $\mathbf{M}$.

The coefficient networks must have enough capacity to produce input-dependent spectral filters comparable to the transformer’s per-layer Q/K projections. Each coefficient network maps $\mathbb{R}^d \rightarrow \mathbb{R}^{K+1}$ —a small MLP with $\sim d \times K$ parameters. For $K = 6$ and $d = 192$, that is $\sim 1,200$ parameters per stage—far fewer than the $2d^2 \approx 74,000$ parameters in a standard attention layer’s Q/K projections. Whether this compression is sufficient is what the experiment tests.

10.10 What This Would Mean for the ORN Programme

The ORN began with the observation that coupling ($\mathbf{M}$) is shared across layers while response (W_V, W_O, FFN) varies. Geometric attention takes the next step: coupling is not a matrix of pairwise weights but a field that emerges from $\mathbf{M}$ ’s geometry. Input-dependent polynomials take the final step: the field is not static but responds to the input, with the spectral depth controlled by a learned coefficient network.

The architecture that follows from the theory is neither a transformer nor a pure operator-space machine. It is a **two-stage coupled system**: operator builds the field (algebraic, structured, shared $\mathbf{M}$), FFN responds to the field (nonlinear, deep, per-stage). Both do real work. The operator carries capacity through input-dependent spectral filtering. The FFN carries capacity through nonlinear transformation. The re-coupling after transformation is the key ingredient

that a pure polynomial lacks—it lets the second coupling see what the first coupling and FFN discovered together.

10.11 The Two-Channel Design and the Self-Measuring Gate

There is a tension in the architecture. If the FFN sees the raw residual stream (including token embeddings), it can bypass the operator entirely—access information that $\mathbf{M}$ never coupled, and render the operator decorative. This is why rank-1 collapse occurs: the FFN learns to do everything itself, removing gradient pressure on $\mathbf{M}$.

But if we constrain the FFN to see *only* the operator’s output, we lose information. $\mathbf{M}$ has rank ~ 70 out of $d = 576$. Language has aspects that are not relational—token frequency, morphological features, rare factual associations. These live in the 83% of representation space orthogonal to $\mathbf{M}$ ’s eigenspace. Projecting through $\mathbf{M}$ discards them.

The resolution: **two channels with a learned gate.**

$$\mathbf{h}_{\text{world}} = \text{FFN}_{\text{world}}(\text{operator output}) \quad (18)$$

$$\mathbf{h}_{\text{direct}} = \text{FFN}_{\text{direct}}(\mathbf{h}_0) \quad (19)$$

$$\mathbf{h}_{\text{out}} = (1 - \sigma(g)) \cdot \mathbf{h}_{\text{world}} + \sigma(g) \cdot \mathbf{h}_{\text{direct}} \quad (20)$$

The world-model channel (eq. 18) forces the FFN to interpret the operator’s coupling. The direct channel (eq. 19) provides raw token features for what the operator cannot represent. The gate $\sigma(g)$ is a function of the operator’s output—it measures whether the world model suffices for this token.

The architecture measures itself. The gate is not added for diagnosis. It IS the architecture. Every forward pass produces both a prediction and a measurement of how much the operator contributed.

- Gate low ($\sigma(g) \approx 0$): the operator handled this token. The world model is sufficient.
- Gate high ($\sigma(g) \approx 1$): the operator couldn’t handle this. Raw features were needed.

After training, the mean gate value is a direct measurement of the operator’s significance:

- Mean gate < 0.3 : the operator handles $> 70\%$ of predictions. It is doing real work.
- Mean gate > 0.7 : the operator handles $< 30\%$. It is weak. The direct channel dominates.

No ablation study needed. No separate experiment. The architecture reports on itself.

The ORN already has this. The perturbative correction gate (bias = -3 , anticorrelating with attention entropy at $r = -0.95$) is exactly this mechanism. It emerged from the architecture, not from a design decision. The gate opens rarely (5% of the time), telling us that the orbital computation handles 95% of predictions. The two-channel design makes this self-measurement explicit and structural.

Design principle. Any architecture that is also a probe is worth pursuing. If the architecture can tell you, during normal inference, how much each component contributes, then you never need to run ablations—the model’s own gates, routing decisions, and channel allocations are the ablation, performed on every token.

11 The Core Bet: Can It Work?

The Orbital Network bets that depth and width are separable. There are exactly two ways this bet can fail: insufficient **capacity**, or failure of **learnability**.

11.1 Capacity: Does the Operator Have Enough Room?

A standard transformer with L layers stores $\mathcal{O}(L \cdot S \cdot d)$ values in its KV cache at inference—that is the “memory” of what computation has been done. The Orbital Network has one $\mathbf{M}$ ($d \times d$) and K polynomial coefficients ($K \times d$). Where does equivalent capacity come from?

The spectral capacity argument. $\mathbf{M}$ ’s eigenvectors form a basis. Each power $\mathbf{M}^k$ amplifies different eigenmodes by different amounts: if $\mathbf{M}$ has eigenvalues $\lambda_1, \dots, \lambda_d$, then $p(\mathbf{M})$ has eigenvalues $p(\lambda_1), \dots, p(\lambda_d)$. A degree- K polynomial can implement K independent spectral filters over this basis. This is K virtual layers of spectral processing—not stored as states but recoverable from the polynomial and $\mathbf{M}$ ’s spectrum.

The KV cache stores $L \cdot S \cdot d$ numbers explicitly. The polynomial stores $K \cdot d$ coefficients and recovers equivalent information by computing $\mathbf{M}^k \mathbf{h}_0$ on demand. If $\mathbf{M}$ has effective rank r (our V2 crystallises to $r \approx 70$ at $d = 576$), the useful information lives in an r -dimensional subspace. The “cache” is implicit in $\mathbf{M}$ ’s spectrum: $r \cdot K$ effective dimensions instead of $L \cdot S \cdot d$ stored values.

The universality guarantee. Orvieto et al. [6] prove that a diagonal linear recurrence followed by a pointwise MLP is a *universal* sequence-to-sequence approximator. The polynomial $p(\mathbf{M})$ is a generalised linear recurrence (its eigenvalues play the role of the recurrence coefficients). The FFN readout is the MLP. Capacity exists in theory.

The honest caveat. The capacity is *different*, not the same. $p(\mathbf{M})\mathbf{h}_0$ can represent any polynomial function of $\mathbf{M}$ ’s eigenvalues applied to $\mathbf{h}_0$ —things that align with $\mathbf{M}$ ’s spectral structure are easy; things that require intermediate nonlinear routing between tokens are hard. The single attention layer handles routing, but only once. A transformer routes L times at different depths, seeing progressively transformed representations.

The question is whether language modelling needs multi-stage routing or single-stage routing. The ORN’s success (matching transformers with shared $\mathbf{M}$) suggests that the coupling rule is depth-invariant—the same $\mathbf{M}$ applies at every layer. If the rule is invariant, multi-stage routing is redundant: one stage with the right operator suffices. But if the ORN’s per-layer FFN and corrections are secretly performing depth-dependent routing, then the single-stage design will fail. This is what the experiments test.

11.2 Learnability: Can Gradient Descent Find Good Operators?

Even if the capacity exists, can SGD navigate operator space? Two concrete concerns:

1. Gradients through $\mathbf{M}^k$. The gradient $\partial\mathcal{L}/\partial\mathbf{M}$ has contributions from each degree:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{M}} = \sum_{k=1}^K c_k \frac{\partial(\mathbf{M}^k \mathbf{h}_0)}{\partial\mathbf{M}} \quad (21)$$

The term $\partial(\mathbf{M}^k \mathbf{h}_0)/\partial\mathbf{M}$ involves k matrix products. For large k , this can vanish or explode. Spectral normalisation ($\mathbf{M} \leftarrow \mathbf{M}/\|\mathbf{M}\|_2$) keeps eigenvalues ≤ 1 , preventing explosion but potentially causing vanishing at high degrees. The polynomial coefficients c_k can compensate by amplifying the signal at each degree, but there is a tension: the normalisation and the coefficients are fighting for control of the eigenvalue magnitudes.

2. Coupled optimisation: $\mathbf{M}$ and coefficients. In ChebNet [9], the graph Laplacian is *fixed*—only the polynomial coefficients are learned. In the Orbital Network, $\mathbf{M}$ is *learned simultaneously* with the coefficients. Changing $\mathbf{M}$ changes the spectral basis, which changes what the coefficients should be. This is a coupled optimisation, harder than either sub-problem alone.

In the ORN, $\mathbf{M}$ crystallises because gradients flow back through L layers of state computation, each providing a different view of what $\mathbf{M}$ should do. In the Orbital Network, $\mathbf{M}$ receives gradient signal only from the polynomial evaluation, one attention layer, and one FFN. Less signal, harder problem.

The argument for learnability. The ORN demonstrates that SGD *can* learn coupling operators. $\mathbf{M}$ crystallises to rank 69–71, locks its asymmetry at 1.41, and stabilises its spectral structure—all via standard backpropagation. The polynomial coefficients add structure to this process: low-degree terms (c_0, c_1) are easy to learn (identity and linear), high-degree terms (c_7, c_8) are harder but contribute less (they operate in $\mathbf{M}$ ’s low-energy spectral tail). The polynomial provides a natural curriculum: learn the easy terms first, refine with harder terms later.

Furthermore, if $\mathbf{M}$ crystallises early (as it does in the ORN by 3B tokens), the coefficient optimisation reduces to a *linear regression in spectral space*: given a fixed basis ($\mathbf{M}$ ’s eigenvectors), find the weights that minimise the loss. Linear regression is the easiest problem in optimisation. The hard part is crystallising $\mathbf{M}$; the easy part is learning what to do with it.

The honest assessment. Capacity is fine in theory. Learnability is the open question. The polynomial parameterisation may provide insufficient gradient signal for $\mathbf{M}$ to crystallise, or the coupled optimisation may converge to a bad local minimum. If $\mathbf{M}$ fails to develop spectral structure in the Orbital Network, the whole programme fails—not because operator-space computation is wrong, but because the parameterisation doesn’t let SGD find the right operator.

The response to this risk is not to hope, but to design the optimiser around the structure of the problem.

12 Optimisation: Three Timescales and a Curriculum

Standard AdamW treats all parameters identically: same learning rate, same momentum, same update rule. But the Orbital Network has three kinds of parameters that are fundamentally different objects operating at different timescales. Treating them identically is like training a brain’s wiring diagram and its synaptic weights at the same speed.

12.1 The Biological Observation

Cortical development separates three timescales of plasticity:

1. **Structural plasticity** (days to weeks): connections form and prune based on activity patterns. The anatomical wiring diagram. Slow, structural, geometry-defining.
2. **Synaptic plasticity** (milliseconds to hours): connection strengths adjust based on local correlations. The functional response. Fast, adaptive, input-dependent.
3. **Critical periods** (months): early development has high plasticity for coarse structure; later development refines fine detail. Coarse before fine. You cannot learn fine phoneme distinctions if you missed the critical period for coarse language structure.

These map directly onto the Orbital Network’s parameters:

Brain	Orbital Network	Timescale	Optimiser
Wiring diagram	$\mathbf{M} = AB^\top$	Geological (slow)	Muon [12]
Synaptic weights	Polynomial coefficients w_k	Fast	AdamW (high lr)
Cortical circuits	Attention + FFN + head	Standard	AdamW (standard lr)
Critical period	Degree curriculum	Training schedule	Scheduler

12.2 Muon for the Operator, AdamW for the Rest

The Muon optimiser [12] was designed for matrix-valued parameters. It orthogonalises the momentum matrix via Newton–Schulz iterations, producing spectrally normalised update directions. Where AdamW updates each element of a matrix independently, Muon updates the matrix *as a geometric object*—the update respects the spectral structure rather than treating it as a bag of scalars.

This is exactly what $\mathbf{M}$ needs:

- $\mathbf{M}$ should update in spectrally clean directions, not distort its eigenstructure randomly.
- Muon’s implicit spectral norm constraint prevents eigenvalue explosion.
- At scale, Muon achieves $2\times$ compute efficiency over AdamW on LLM pretraining [13] and has been deployed at 355B+ parameters.

The optimiser split:

Parameters	Optimiser	Learning rate	Weight decay
A, B (the operator)	Muon	1×10^{-4}	0.01
w_k (polynomial coefficients)	AdamW	3×10^{-3}	0.0
Attention, FFN, embeddings, head	AdamW	3×10^{-4}	0.1

$\mathbf{M}$ learns $30\times$ slower than the polynomial coefficients. The coefficients have no weight decay—they are scaling factors, not connection weights, and should be free to take any value. The Muon optimiser ensures that $\mathbf{M}$ ’s updates preserve spectral structure; AdamW ensures the coefficients adapt quickly.

12.3 Degree Curriculum

Do not train all K polynomial degrees from step 1. Start with $K = 1$ (identity + linear $\mathbf{M}$), then gradually unlock higher degrees:

Steps	Active degrees
0–1,000	$K = 1$: just $c_0I + c_1\mathbf{M}$
1,000–3,000	$K = 3$: $+\mathbf{M}^2, \mathbf{M}^3$
3,000–6,000	$K = 5$: $+\mathbf{M}^4, \mathbf{M}^5$
6,000+	$K = 8$: full polynomial

Why. Low-degree terms dominate the polynomial (they correspond to the large eigenvalues of $\mathbf{M}$). Training $\mathbf{M}$ against a degree-1 objective is simple: learn the best linear coupling. Once $\mathbf{M}$ has crystallised its top eigenmodes against this simple objective, higher degrees refine the tail of the spectrum. Each new degree adds a finer spectral filter on top of an already-stable base.

This is a curriculum in operator space, not data space. The “easy examples” are low polynomial degrees; the “hard examples” are high degrees. $\mathbf{M}$ is never asked to solve the full polynomial problem from random initialisation—it builds up to it, one spectral level at a time.

The critical period analogy. In neurodevelopment, cortical connectivity forms coarse structure first (long-range connections, low spatial frequency) and fine structure later (local circuits, high spatial frequency). Missing the critical period for coarse structure means fine structure

never develops correctly. The degree curriculum enforces the same ordering: $\mathbf{M}$ ’s coarse spectral structure (top eigenmodes, low polynomial degree) must crystallise before fine structure (tail eigenmodes, high degree) is introduced.

12.4 Spectral Crystallisation Pressure

In the ORN, $\mathbf{M}$ crystallises spontaneously—the loss landscape pushes it toward low-rank structure. In the Orbital Network, with less gradient signal (one polynomial evaluation instead of L layers), $\mathbf{M}$ may not crystallise on its own. An explicit regularisation term can provide the pressure:

$$\mathcal{L}_{\text{reg}} = \lambda_{\text{rank}} \sum_{i=2}^d \frac{\sigma_i}{\sigma_1} + \lambda_{\text{stable}} \|\boldsymbol{\sigma} - \boldsymbol{\sigma}_{\text{prev}}\| \quad (22)$$

where σ_i are the singular values of $\mathbf{M}$. The first term penalises high effective rank (pushes toward low-rank structure). The second term penalises rapid spectral change (pushes toward stability). Together, they encode: “crystallise, and once crystallised, stay crystallised.”

Both terms become inactive once $\mathbf{M}$ has crystallised: a low-rank, spectrally stable $\mathbf{M}$ incurs near-zero penalty. The regularisation provides pressure during the critical period and releases it afterward.

Connection to continual learning. Spectral regularisation for neural networks has precedent: keeping the maximum singular value of each layer near 1 improves training stability and prevents catastrophic forgetting in continual learning [14]. Our application is different—we want to *encourage* low rank, not preserve full rank—but the mechanism is the same: control the eigenspectrum to control learning dynamics.

13 How a Transformer Understands Stories vs How an Operator Would

Consider a model trained on TinyStories processing the prefix:

Once upon a time there was a little girl named

Both a transformer and an operator-space model must predict that “Lily,” “Emma,” or “Sophie” are likely, while “the” or “17” are not. They arrive at this prediction through fundamentally different computations.

13.1 The Transformer: Brute-Force State Accumulation

A transformer processes this prefix through L layers of interleaved attention and FFN. At each layer, the state at every position is updated: attention mixes information between tokens, FFN transforms each token’s representation. After L layers, the state at position 10 has accumulated enough context—from “Once” through “named”—to place high probability on names.

The knowledge that “named” is followed by a name is not stored in any single place. It is distributed across L layers of attention patterns and FFN weights. There is no location where you can point and say “here is the model’s understanding of story structure.” The world model is implicit, smeared across $\mathcal{O}(L \cdot d^2)$ parameters, and irreducibly sequential: you cannot access layer 8’s representation without computing layers 1–7 first.

This works. It works at scale, it works on diverse tasks, it produces fluent text. But it is brute force: the same L -step computation runs for “the cat sat” (trivial) and “the cat that the dog that the man owned chased” (centre-embedded recursion requiring genuine depth).

13.2 The Operator: Attractor Basins in Spectral Space

An operator-space model would work differently. $\mathbf{M}$'s eigenstructure defines a landscape—a space with basins, ridges, and valleys. Training shapes this landscape so that:

- **Character introduction** is an attractor basin. After “there was a little [noun],” the state flows toward a region where names, descriptions, and attributes are geometrically accessible. Not specific names—the *structural slot* for a name.
- **Conflict initiation** is another basin. After a character is established, $\mathbf{M}$'s geometry warps so that “but,” “one day,” and “suddenly” become accessible—not because the model memorised these words, but because $\mathbf{M}$'s spectral structure creates a valley the state flows into.
- **Resolution** is a third basin. After conflict, the geometry channels the state toward “and they lived” / “she was happy” / “he learned that.”

The model does not store “character $\rightarrow$ conflict $\rightarrow$ resolution” as a sequence of states. $\mathbf{M}$'s eigenstructure *defines* these basins. The polynomial $p(\mathbf{M})$ moves the state between basins. The degree controls how many narrative transitions the model can represent: degree 1 = one transition, degree 3 = three transitions. Story structure is encoded in $\mathbf{M}$'s spectrum, not in a chain of memorised states.

13.3 Comprehension and Imagination

When the model reads “Once upon a time there was a,” two things happen simultaneously:

Comprehension. The state is pulled toward the appropriate basin. $\mathbf{M}$ acts as a lens: it resolves the diffuse input embedding into a structured region of representation space. Reading is the operator decomposing the input into its spectral components—projecting the prefix onto $\mathbf{M}$'s eigenbasis to determine which narrative pattern is active.

Imagination. Before the next token arrives, the operator has already shaped the space of what *could* come next. “Little girl,” “big bear,” “kind old man”—all of these live in the same attractor basin. The model has not predicted one token; it has prepared a *landscape* of likely continuations. The landscape is the imagined space. Generation means sampling from this landscape.

In a transformer, comprehension and imagination are the same operation—the forward pass both processes what came before and sets up what comes next. In operator space, they separate:

- **Comprehension** = projecting the input onto $\mathbf{M}$'s eigenbasis. Which basin does this prefix activate?
- **Imagination** = the shape of the basin itself. What continuations does the basin make geometrically accessible?

The polynomial coefficients control the *depth* of imagination. Degree 1 = predict the next immediate token. Degree 8 = anticipate narrative arcs that haven't started yet.

13.4 The Bootstrapping Problem

At the start of training, $\mathbf{M}$ is random—no basins, no structure. The landscape is flat. But the model can do shallow comprehension: bigrams, trigrams, simple co-occurrences. These gradients slowly shape $\mathbf{M}$, creating the first attractor basins.

As $\mathbf{M}$ crystallises, the basins deepen. The model starts to “expect” certain structures—not specific words but narrative patterns. The crystallisation IS the bootstrapping. $\mathbf{M}$ goes from a flat random landscape to a structured one with valleys (common patterns), ridges (transitions), and peaks (impossible sequences).

The corrections enter here. They handle token-specific details that the basins cannot capture. The basin says “a name goes here.” The correction says “specifically Lily, because preceding tokens included feminine cues.” The operator provides narrative grammar; the corrections provide lexical specifics.

13.5 The Gap

A transformer does all of this—comprehension, imagination, narrative structure—but through L layers of sequential state transformation. The knowledge is there but not *in* any one place.

The operator-space claim is that this knowledge CAN be concentrated into $\mathbf{M}$ ’s spectral structure, because the narrative grammar is:

- Low-dimensional (rank 70 of 576 in V2—only 70 eigenmodes carry coupling structure).
- Layer-invariant (spectral correlation ~ 0.98 —the same coupling at every depth).
- Task-transferable (+0.73% frozen- $\mathbf{M}$ gap—the grammar works across datasets).

The Krylov measurement (Section 16) asks: does the transformer’s state trajectory stay close to $\mathbf{M}$ ’s algebraic subspace? The attractor basin picture predicts yes—because moving between basins is a transition defined by $\mathbf{M}$ ’s eigenvectors, and $\mathbf{M}$ ’s powers ARE these transitions. c_0 (stay in current basin) + $c_1\mathbf{M}$ (one transition) + $c_2\mathbf{M}^2$ (two transitions) + ...

But this only works if the basins are *linear* in $\mathbf{M}$ ’s eigenbasis. If the basins are defined by nonlinear interactions between eigenvectors—if the FFN at each layer is what creates the basin structure, not $\mathbf{M}$ —then the Krylov subspace misses them. The per-layer nonlinearity is not correction; it is structural. And the operator-space programme fails.

That is what the measurement determines. Not a theoretical argument—an empirical one.

14 Relational Templates and Metaphor Composition

Consider the sentence: “The committee’s recommendation was subsequently vindicated by independent analysis.” A naïve analysis labels this as high-information: long sentence, complex syntax, abstract vocabulary. But look at what “vindicate” implies. The word alone activates:

- Three parties: a claimant, a challenger, and an arbiter.
- A temporal sequence: claim $\rightarrow$ challenge $\rightarrow$ vindication.
- A judgment: the original claim was correct.

The rest of the sentence is slot-filling. “Committee” fills the claimant slot. “Board” fills the challenger slot. “Independent analysis” fills the arbiter slot. “Subsequently” makes the implied temporal order explicit—it is padding, not information.

The sentence is low-information, high-structure. A single word carries the relational template; the remaining words fulfil roles within it.

14.1 Words as Operators

This observation generalises. Most of language is structure, not content. Words are not atomic symbols to be predicted—they are compressed relational templates:

- “Vindicate” $\rightarrow$ three-party judgment template
- “Despite” $\rightarrow$ expectation-violation template
- “Because” $\rightarrow$ causal chain template
- “Although” $\rightarrow$ concession-counterpoint template

Each word, when read, *wraps the coupling field* in a specific way. “Vindicate” opens three relational slots and establishes temporal ordering. “Despite” creates a tension between an expectation and its violation. The word IS the operator. The embedding of a word is its operator signature—how it transforms $\mathbf{M}$ ’s coupling geometry.

14.2 Nested Composition: Metaphors All the Way Down

Relational templates are not flat. They compose hierarchically, in the sense of Lakoff and Johnson’s *Metaphors We Live By* [37]. “Argument is war”—we *attack* positions, *defend* claims, *shoot down* ideas. This is not decorative language; it is the structure of thought. And it nests:

1. **Spatial templates** (in/out, up/down, near/far)—embodied, universal.
2. **Force templates** (push, pull, resist, yield)—built on spatial + causation.
3. **Container templates** (inside/outside, boundary, containment)—spatial + boundary.
4. **Abstract templates** (judgment = container + agent + temporal; causation = force + sequence; argument = conflict + container + resolution).

Each level is a relational template composed from simpler templates. “Vindicate” sits at level 4: judgment (level 3) + social role + emotional valence. “Cat” sits at level 2: spatial + animate + small.

The polynomial degree as nesting depth. This hierarchy maps directly onto the polynomial:

- Degree 1: $c_1 \mathbf{M} \mathbf{h}_0$ —basic spatial/relational templates.
- Degree 2: $c_2 \mathbf{M}^2 \mathbf{h}_0$ —composed templates (two relational steps).
- Degree 3: $c_3 \mathbf{M}^3 \mathbf{h}_0$ —abstract templates (three levels of composition).
- Degree K : $c_K \mathbf{M}^K \mathbf{h}_0$ — K levels of metaphor nesting.

$\mathbf{M}^1$ gives basic relational structure. $\mathbf{M}^2$ composes templates. $\mathbf{M}^3$ composes the compositions.

The polynomial is composing metaphors through $\mathbf{M}$ ’s algebra. The degree is the nesting depth of conceptual structure.

The input-dependent coefficients (Section 10.6) are now interpretable: the coefficient network reads each word and selects which nesting depth to activate. “Cat” → low degrees (simple spatial template). “Vindicate” → high degrees (deep metaphorical composition). The coefficients are a **template selector**, not a complexity estimator.

14.3 Implications for $\mathbf{M}$ ’s Rank

If words are compositions of relational templates, how many fundamental templates does $\mathbf{M}$ need to encode?

Many words share the same base templates:

- “Vindicate,” “exonerate,” “absolve” → same three-party judgment template.
- “Despite,” “although,” “notwithstanding” → same expectation-violation template.
- “Because,” “since,” “therefore” → same causal template.

The number of *unique* fundamental templates may be small—perhaps 50–100 base relational patterns from which all others compose. $\mathbf{M}$ ’s effective rank of 69–71 (at $d = 576$, V2) aligns with this: **one eigenmode per fundamental template**.

The capacity prediction. At $d = 192$ (our experiments), $\mathbf{M}$ crystallised to rank ~ 25 . This is insufficient for the full template vocabulary—only the most common patterns fit. The 36% Krylov recovery may be low not because language is mostly content, but because $\mathbf{M}$ was **too small** to encode all the relational templates it needed.

At $d = 576$ (V2 scale), rank 69–71 accommodates more templates. At $d = 2048$ (V3 scale), the rank should increase further. **Prediction: Krylov recovery increases with d** —not because the polynomial computes more, but because $\mathbf{M}$ has room for more templates. If most of language is relational structure (and we argue it is), then at sufficient d , the operator captures the majority of the computation.

This is testable: run the Krylov measurement at $d \in \{64, 128, 192, 384\}$ and measure recovery as a function of model dimension. If recovery scales with d , the operator programme has room

to grow. If it saturates at 36% regardless of d , then 64% of language genuinely requires per-step nonlinear processing.

15 Status of the Conjectures

An honest assessment based on all experiments to date.

Conjecture 1 (Structural Invariance): STRONG. SharedM works—the ORN matches or beats per-layer coupling at matched parameters. Spectral invariance (~ 0.98 across five architecture families) is stronger evidence for a universal coupling kernel than anything in the Platonic Representation literature. Frozen- $\mathbf{M}$ transfers (+0.73% gap). Wide shared FFN beats per-layer FFN. Literature supports this from multiple directions (Koopman theory, compositional generalisation, geometric deep learning). This conjecture is the most established part of the programme.

Conjecture 2 (Operator Representation): MIXED. $\mathbf{M}$ has composability and spectral decomposition by construction—any matrix generates a semigroup (Hille–Yosida). But we have not yet shown that the *computation* uses these properties. Polynomial degrees above 1 do not activate in our experiments. $\mathbf{M}$ collapses to rank 1 when unconstrained. The Lie–Trotter connection (ORN layer structure = operator splitting) is mathematically precise, and the commutator $[\mathbf{M}, \text{FFN}]$ drops from rank 150 to 65 during training—suggesting the coupling and response become more commutative as $\mathbf{M}$ crystallises. Whether the spectral modes correspond to interpretable relational templates remains untested (eigenmode probing experiment pending).

Conjecture 3 (Operator Recovery): WEAK. Krylov recovery is 36% (residual 92%). A polynomial of $\mathbf{M}$ in embedding space is just a linear map—zero additional capacity. Even with input-dependent coefficients and constrained rank, the two-stage architecture (val 3.93) underperforms a simple one-layer ORN with deep FFN (val 3.80). The polynomial of the *coupling pattern* C (multi-hop attention) is a more promising direction—it provides genuinely compositional depth that single-hop + FFN cannot express—but this has not yet been tested.

Best confirmed architecture. One $\mathbf{M}$ -coupled attention layer ($Q = \mathbf{h}A$, $K = \mathbf{h}B$, same A, B as SharedM) plus four FFN layers. Val 3.80 at $d = 192$ on TinyStories, 12K steps. This is a one-layer ORN with more FFN. No operator algebra, no polynomial depth, no multi-hop. The SharedM coupling is doing real work; everything else we added was decorative.

Most promising unconfirmed architecture. The two-channel multi-hop design with self-measuring gate:

1. Compute coupling $C = \text{softmax}((\mathbf{h}A)(\mathbf{h}B)^\top / \sqrt{d})$ using SharedM.
2. Multi-hop: $\mathbf{h}_{\text{op}} = \sum c_k C^k V$ (polynomial of the coupling *pattern*, not of $\mathbf{M}$).
3. World channel: $\text{FFN}_{\text{world}}(\mathbf{h}_{\text{op}})$ (interpret the multi-hop output).
4. Direct channel: $\text{FFN}_{\text{direct}}(\mathbf{h}_0)$ (raw features for what the operator misses).
5. Gate: $\sigma(g(\mathbf{h}_{\text{op}}))$ measures operator contribution per token.

If multi-hop beats single-hop and the gate assigns $> 50\%$ weight to the world channel, the operator is doing non-trivial compositional work. This experiment is pending.

16 Experiment Log: What We Found

This section records findings from the experimental programme as they arrive. It is a working log, not a polished results section.

16.1 Group II: Six Operator-Space Architectures (5K steps, d=192)

Six models trained on TinyStories at matched scale. The standard 8-layer ORN is the baseline. Five operator-space candidates replace the 8 layers with different algebraic computations plus a single attention layer and two FFN layers.

Model	Mechanism	Val Loss	Gap
A. StandardORN (8 layers)	State-space baseline	4.355	—
B. MatExpORN	$\exp(\alpha \mathbf{M}) \cdot \mathbf{h}_0$	4.589	+5.4%
C. SpectralORN	Per-eigenmode scaling	4.740	+8.8%
D. PolyORN	$\sum c_k \mathbf{M}^k \mathbf{h}_0$	4.614	+6.0%
E. S4ORN	Depth convolution via $\mathbf{M}$	4.668	+7.2%
F. DeferredORN	Composed low-rank updates	4.638	+6.5%

All operator-space models are 5–9% worse than the baseline. None collapse to random. MatExpORN is closest at 5.4% gap, but learned $\alpha = 0.134$ —so small that $\exp(\alpha \mathbf{M}) \approx I + 0.134\mathbf{M}$, effectively a linear layer. PolyORN’s coefficients appeared stuck at $c_0 = 1, c_1 = 0$ (a logging bug: the model used per-dimension weights, not scalar coefficients, and the logged parameter was unused).

Assessment. The operator-space models work (they learn, they generate text) but they don’t match 8-layer depth. The single attention layer + two FFN layers are doing most of the work. The operator contribution is unclear—it may be decorative.

16.2 OrbitalNetV2: The Clean Test (12K steps, d=192)

A redesigned architecture incorporating three innovations: M-coupled attention ($\mathbf{Q} = \mathbf{h} @ \mathbf{A}$, $\mathbf{K} = \mathbf{h} @ \mathbf{B}$, using the same $\mathbf{A}$, $\mathbf{B}$ that define $\mathbf{M}$ —no separate coupling), degree curriculum ($\mathbf{K}=1 \rightarrow 3 \rightarrow 5 \rightarrow 8$ over training), and two-timescale learning rates ($\mathbf{M}$ at 10^{-4} , polynomial coefficients at 3×10^{-3}).

Model	Config	Val Loss	vs Baseline
A. StandardORN	8 layers, 5K steps	4.355	—
B. OrbitalNetV2	Curriculum + two-timescale, 12K	3.822	−12.2%
C. OrbitalPlain	No innovations, 12K	4.08*	−6.3%
D. LinearControl	Plain W (no M), 12K	pending	—

*C still running at time of writing.

The headline. OrbitalNetV2 *beats* the 8-layer baseline by 0.53 nats. The curriculum and two-timescale learning contribute ~ 0.26 nats (B vs C).

The concern: rank-1 collapse. In all OrbitalNet variants, $\mathbf{M}$ collapsed to effective rank 1. This happened with and without spectral regularisation, confirming it is intrinsic, not induced.

Rank-1 $\mathbf{M} = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top$ means every power $\mathbf{M}^k$ is also rank 1, along the same direction. The polynomial $p(\mathbf{M})\mathbf{h}_0$ reduces to a scalar polynomial applied to the projection of $\mathbf{h}_0$ onto $\mathbf{v}_1$. All eight degrees of the polynomial collapse to one effective degree of freedom.

This raises the question: is the operator providing real algebraic depth, or is it a rank-1 projection followed by attention + FFN doing all the work? The loss improvement (3.82 vs 4.35) is real, but may reflect the longer training (12K vs 5K steps) or the M-coupled attention (which uses $\mathbf{M}$ for $\mathbf{Q}/\mathbf{K}$, providing a learned coupling geometry even at rank 1), not the polynomial depth.

What rank-1 means for capacity. With rank-1 $\mathbf{M}$, the Krylov subspace $\mathcal{K}(\mathbf{M}, \mathbf{h}_0)$ is at most 2-dimensional: $\text{span}\{\mathbf{h}_0, \mathbf{M}\mathbf{h}_0\}$. Higher powers $\mathbf{M}^k\mathbf{h}_0$ are scalar multiples of $\mathbf{M}\mathbf{h}_0$ —they add no new dimensions. A polynomial of any degree reduces to $\mathbf{h} \approx c_0\mathbf{h}_0 + c_1\mathbf{M}\mathbf{h}_0$: identity plus a learned linear projection.

For the operator to provide genuine depth—for degree K to add information that degree $K-1$ cannot— $\mathbf{M}$ needs rank $\geq K$. The rank-1 collapse means the polynomial cannot access deeper computation even though the parameters exist to do so.

Why rank-1? Two hypotheses:

1. **The task is too easy.** TinyStories at $d=192$ may not require multi-layer processing. A rank-1 projection + attention + FFN suffices. $\mathbf{M}$ collapses because higher rank adds nothing useful.
2. **The optimisation landscape favours collapse.** Spectral normalisation ($\mathbf{M} \leftarrow \mathbf{M}/\|\mathbf{M}\|_2$) keeps the top eigenvalue at 1 while higher eigenvalues contribute vanishing gradients ($\mathbf{M}^k$ decays for modes with $|\lambda_i| < 1$). The fast polynomial coefficients adapt to whatever rank-1 $\mathbf{M}$ provides, removing gradient pressure on $\mathbf{M}$ to develop higher-rank structure.

Both may be true simultaneously. The next experiment (Krylov recovery measurement) will distinguish them: if the 8-layer ORN’s state trajectory lives in a 2D Krylov subspace, the task is genuinely easy and rank 1 is correct. If the trajectory requires a higher-dimensional subspace, the rank-1 collapse is a training failure and the optimisation strategy needs revision.

16.3 The Krylov Recovery Measurement (Result)

The foundational test. Given a trained ORN ($d=192$, $L=8$, 5K steps, TinyStories), project the final state $\mathbf{h}_L$ onto the Krylov subspace $\mathcal{K}(\mathbf{M}, \mathbf{h}_0) = \text{span}\{\mathbf{h}_0, \mathbf{M}\mathbf{h}_0, \mathbf{M}^2\mathbf{h}_0, \dots\}$ and measure the residual.

Degree K	Recovery	Residual	
0	0.110	0.984	
1	0.178	0.974	
3	0.241	0.961	
5	0.286	0.949	
8	0.340	0.930	
10	0.358	0.922	← saturation
16	0.360	0.921	

The Krylov subspace recovers only 36% of the orbit. Residual: 92%. The state trajectory massively escapes $\mathbf{M}$ ’s algebraic structure. Adding polynomial degrees beyond 10 contributes nothing—the subspace saturates. $\mathbf{M}$ has effective rank 25, but the useful Krylov dimension is approximately 10.

Per-layer analysis reveals why. Corrections grow dramatically with depth:

Layer	Displacement	Correction	Corr/Disp
1	1.37	0.10	7%
2	1.48	0.06	4%
3	1.42	0.08	6%
4	1.84	0.14	8%
5	1.57	0.25	16%
6	2.02	0.49	24%
7	2.46	0.83	34%
8	2.89	1.43	50%

Layers 1–4: corrections are 4–8% of displacement. Genuinely perturbative. Layers 5–6: corrections are 16–24%. Significant but secondary. Layers 7–8: corrections are 34–50%. **Load-bearing.**

Overall $\rho = 0.34$ —in the contested zone, not perturbative (< 0.15) but not dominant (> 0.50).

Interpretation: the right decomposition, not a failure. The initial reading of this result is negative: the Krylov subspace misses 64% of the orbit. But look at what the per-layer data actually shows. Layers 1–4 are 92–96% linear in $\mathbf{M}$. The operator *dominates* the early computation—the coupling geometry, the “what relates to what.” Then layers 5–8 require increasing nonlinearity: the FFN and corrections are building on top of the structure that $\mathbf{M}$ established.

This is the coupling-response decomposition working exactly as designed. $\mathbf{M}$ builds the relational structure. FFN responds to it. The FFN doing real work is not a failure—it is correct, as long as it is *responding to a rich field* rather than building the field from scratch.

The difference between architectures:

- **Transformer:** every layer builds AND responds. Attention computes coupling, FFN transforms state, next layer starts over. The “world model” is never a separable object—it is smeared across all L layers.
- **What the data suggests:** $\mathbf{M}$ builds the coupling field (layers 1–4, 92–96% linear). FFN responds to it (layers 5–8, 50–76% nonlinear). In the current ORN these are interleaved. In the right architecture they would be sequential: build the field first, respond to it second.

The question is not “can $p(\mathbf{M})$ replace all L layers?” (no—the 92% residual proves this). The question is: **can $p(\mathbf{M})$ build a coupling field rich enough that a nonlinear readout stack matches L -layer interleaved processing?**

This is a much more reasonable claim. The polynomial builds the geometry; the FFN navigates it. Both do real work, but they do *different* work. The operator-space contribution is not the entire computation—it is the *structural* computation, the world model that the nonlinear layers react to.

The architectural implication. The OrbitalNetV2 had two FFN layers as readout. The Krylov data says: the nonlinear response needs to be *deep*—at least 4 layers of FFN to match what layers 5–8 of the ORN contribute. But crucially, these FFN layers do not need their own attention. They respond to the coupling field that $p(\mathbf{M})$ already built.

The architecture becomes:

1. **Build the field:** $\mathbf{h}_w = p(\mathbf{M})\mathbf{h}_0$ (algebraic, composable, $K \approx 4$ –8).
2. **Geometric coupling:** inner products in warped space define which tokens relate (Section 10).
3. **Deep nonlinear response:** 4+ layers of FFN operating on the field-coupled representation. No additional attention—the coupling is already established. The FFN stack interprets and transforms, it does not re-couple.

This is not operator-space replacing state-space. It is operator-space building the world model, and state-space responding to it. The operator provides the geometry; the FFN provides the content.

The rank-1 collapse, revisited. The OrbitalNetV2’s rank-1 collapse was initially attributed to shallow readout—the hypothesis being that with only two FFN layers, the model could not exploit high-rank $\mathbf{M}$ structure, so $\mathbf{M}$ collapsed.

The geometric attention experiment **falsifies this hypothesis**. GeometricDeep (4 FFN layers) shows identical rank-1 collapse: same result as GeometricShallow (2 FFN layers), same val loss (3.900 vs 3.899). Doubling response depth adds nothing—literally zero improvement.

This means the bottleneck is not response capacity. It is the coupling. More FFN is like chewing the same food longer: if the coupling did not bring the right information together, no amount of FFN processing will fix it.

16.4 Geometric Attention Results

Model	Config	Val Loss	vs V2
B. GeometricDeep	Symm. coupling + 4 FFN	3.900	+0.078
C. GeometricShallow	Symm. coupling + 2 FFN	3.899	+0.077
D. ExplicitDeep	M-coupled attn + 4 FFN	pending	—
ref: OrbitalNetV2	M-coupled attn + 2 FFN	3.822	—
ref: StandardORN	8-layer state-space	4.355	—

Finding: coupling is the bottleneck, not response depth. GeometricShallow (2 FFN) = GeometricDeep (4 FFN) within noise. This confirms the V2 shared-FFN finding at a new level: one

wide FFN suffices for response. The scarce resource is coupling—how many times the model can re-evaluate which tokens relate to which. The commodity is transformation—FFN processing of coupled representations.

This directly motivates the two-stage architecture: the value is not in deeper FFN but in **re-coupling**—seeing the FFN-transformed state and routing again through $\mathbf{M}$ ’s geometry.

Rank-1 collapse persists. Both geometric models collapse $\mathbf{M}$ to rank 1, despite 4 FFN layers of response capacity. The next experiment (two-stage with asymmetric coupling and $A, B \in \mathbb{R}^{d \times r}$) constrains rank from below: $\mathbf{M}$ cannot collapse below r . If constraining rank forces the polynomial to use multiple eigenmodes AND improves quality, the rank-1 collapse was a training failure. If quality is unchanged, rank 1 is what the task genuinely needs at this scale.

References

- [1] A. Gu, K. Goel, C. Ré, “Efficiently modeling long sequences with structured state spaces,” *ICLR*, 2022.
- [2] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv:2312.00752*, 2023.
- [3] I. Schlag, K. Irie, J. Schmidhuber, “Linear transformers are secretly fast weight programmers,” *ICML*, 2021.
- [4] S. Bai, J. Z. Kolter, V. Koltun, “Deep equilibrium models,” *NeurIPS*, 2019.
- [5] J. Halverson, A. Maiti, K. Stoner, “Neural networks and quantum field theory,” *Machine Learning: Science and Technology*, 2021.
- [6] A. Orvieto et al., “Universality of linear recurrences followed by non-linear projections,” *arXiv:2307.11888*, 2023.
- [7] T. Fischbacher and L. Comsa, “Intelligent matrix exponentiation,” *arXiv:2008.03936*, 2020.
- [8] Z. Li, N. Kovachki et al., “Fourier neural operator for parametric partial differential equations,” *ICLR*, 2021.
- [9] M. Defferrard, X. Bresson, P. Vandergheynst, “Convolutional neural networks on graphs with fast localized spectral filtering,” *NeurIPS*, 2016.
- [10] R. Levie, F. Monti et al., “CayleyNets: Graph convolutional neural networks with complex rational spectral filters,” *IEEE Trans. Signal Processing*, 2019.
- [11] S. M. Jayakumar et al., “Multiplicative interactions and where to find them,” *ICLR*, 2020.
- [12] K. Jordan, “Muon: An optimizer for hidden layers in neural networks,” 2024. <https://kellerjordan.github.io/posts/muon/>
- [13] Z. Liu et al., “Muon is scalable for LLM training,” *arXiv:2502.16982*, 2025.
- [14] M. Titsias, J. Schwarz, A. G. de G. Matthews, R. Pascanu, Y. W. Teh, “Learning continually by spectral regularization,” *ICLR*, 2025.
- [15] R. Csordás, K. Irie, J. Schmidhuber, “The devil is in the detail: Simple tricks improve systematic generalization of transformers,” *EMNLP*, 2021.
- [16] M. Huh, B. Cheung, T. Wang, P. Isola, “The Platonic representation hypothesis,” *ICML*, 2024.
- [17] Relaxed Recursive Transformers, *ICLR*, 2025.
- [18] M. Bronstein, J. Bruna, T. Cohen, P. Veličković, “Geometric deep learning: Grids, groups, graphs, geodesics, and gauges,” *arXiv:2104.13478*, 2021.
- [19] N. Muennighoff et al., “OLMoE: Open mixture-of-experts language models,” 2024.
- [20] S. Brunton et al., “Modern Koopman theory for dynamical systems,” *SIAM Review*, 2022.
- [21] B. Lusch, J. N. Kutz, S. Brunton, “Deep learning for universal linear embeddings of nonlinear dynamics,” *Nature Communications*, 2018.

- [22] M. Colbrook et al., “The limits of Koopman learning,” 2024.
- [23] K.-J. Engel and R. Nagel, *One-Parameter Semigroups for Linear Evolution Equations*, Springer, 2000.
- [24] H. Trotter, “On the product of semi-groups of operators,” *Proc. AMS*, 1959.
- [25] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*, SIAM, 1997.
- [26] G. Vidal, “Entanglement renormalization,” *Physical Review Letters*, 2007.
- [27] Y. Levine et al., “Quantum entanglement in deep learning architectures,” *Physical Review Letters*, 2019.
- [28] A. Ali et al., “What can linear attention do? A theoretical analysis,” 2024.
- [29] S. Arora et al., “Simple linear attention language models balance the recall-throughput tradeoff,” 2024.
- [30] G. Deletang et al., “Neural networks and the Chomsky hierarchy,” *ICLR*, 2023.
- [31] M. Koch-Janusz and Z. Ringel, “Mutual information, neural networks and the renormalization group,” *Nature Physics*, 2018.
- [32] B. van Nierop, “Neural networks as gauge theories,” 2024.
- [33] R. Baxter, *Exactly Solved Models in Statistical Mechanics*, Academic Press, 1982.
- [34] S. White, “Density matrix renormalization group algorithms with a single center site,” *Physical Review B*, 2005.
- [35] D. Roberts, S. Yaida, B. Hanin, *The Principles of Deep Learning Theory*, Cambridge UP, 2022.
- [36] P. Bak, C. Tang, K. Wiesenfeld, “Self-organized criticality: An explanation of the $1/f$ noise,” *Physical Review Letters*, 1987.
- [37] G. Lakoff and M. Johnson, *Metaphors We Live By*, University of Chicago Press, 1980.
- [38] Sirsh, “Orbital Response Networks: Shared coupling geometry for efficient language modelling,” *Working paper*, 2026.

A Meta-Attention: An Operator That Produces Attention

All previous designs either compute the coupling field from scratch at each layer (the transformer) or try to replace it with a one-shot computation (our failed experiments). The fundamental issue: one-shot coupling is too rigid (can’t adapt to context), per-layer recomputation is too expensive (can’t share geometry).

Meta-attention resolves this by separating the geometry into two components:

$$\mathbf{M}_{\text{effective}} = \underbrace{\mathbf{M}_{\text{base}}}_{\text{the grammar}} + \underbrace{\Delta\mathbf{M}(\text{context})}_{\text{the warp}} \quad (23)$$

$\mathbf{M}_{\text{base}} = AB^\top$ is the learned grammar—the coupling geometry crystallised during training, shared across all positions and layers. This is what the ORN already learns.

$\Delta\mathbf{M} = \sum_i \mathbf{u}(x_i)\mathbf{v}(x_i)^\top$ is the **accumulated context warp**—each token contributes a rank-1 update to the geometry. “The” barely warps it. “Vindicate” warps it significantly, opening three-party coupling channels. The accumulated warp IS the world model state—the context built from reading.

The coupling pattern falls out:

$$C = \text{softmax}\left(\frac{\mathbf{h}\mathbf{M}_{\text{effective}}\mathbf{h}^\top}{\sqrt{d}}\right) = \text{softmax}\left(\frac{\mathbf{h}\mathbf{M}_{\text{base}}\mathbf{h}^\top + \sum_i (\mathbf{h}\mathbf{u}_i)(\mathbf{v}_i^\top\mathbf{h}^\top)}{\sqrt{d}}\right) \quad (24)$$

The first term is the base coupling (the grammar). The second term is the context-dependent warp (what you’ve read). This is linear attention’s fast-weight update *added to* a base coupling from $\mathbf{M}$, not replacing it.

Why “meta-attention.” Standard attention computes a coupling pattern from Q, K projections. Meta-attention computes the *geometry* from which the coupling pattern is derived. The meta-level ($\mathbf{M}_{\text{base}} + \Delta\mathbf{M}$) generates the object-level (C). The meta-level has all the properties we conjecture:

- **Composition:** $\Delta\mathbf{M}_A + \Delta\mathbf{M}_B = \Delta\mathbf{M}_B + \Delta\mathbf{M}_A$ (warps accumulate commutatively).
- **Continuity:** the warp is a smooth function of the token embeddings.
- **Spectral evolution:** each token’s rank-1 warp shifts $\mathbf{M}$ ’s eigenvalues, modifying which coupling modes are active.
- **Self-measuring:** $\|\Delta\mathbf{M}\|_F / \|\mathbf{M}_{\text{base}}\|_F$ measures how much context has warped the base geometry.

What the FFN does. The FFN interprets the coupling that $\mathbf{M}_{\text{effective}}$ produces. It sees the warped-geometry attention output and extracts predictions. The geometry ($\mathbf{M}_{\text{base}} + \Delta\mathbf{M}$) is the world model. The FFN lives in it.

How this differs from existing architectures.

Architecture	Coupling	Geometry
Transformer	$W_Q W_K^\top$ (per-layer, no decomposition)	Implicit, not separated
Linear attention	$\sum k_i v_i^\top$ (context only, no base)	No base grammar
ORN (SharedM)	$AB^\top$ (shared base, no context warp)	Fixed base, no adaptation
Meta-attention	$AB^\top + \sum u_i v_i^\top$ (base + warp)	Separated, compositional

The transformer has coupling without decomposition. Linear attention has context without grammar. The ORN has grammar without context adaptation. Meta-attention has both: a learned grammar that adapts to context through compositional warps.

Implementation in reduced space. For efficiency, the warp operates in r -dimensional space. Queries and keys project through $A, B \in \mathbb{R}^{d \times r}$ (the base geometry). Warp keys and values project through separate $P_u, P_v \in \mathbb{R}^{d \times r}$ (the warp generators). The accumulated warp $W_j = \sum_{i \leq j} \mathbf{w} \mathbf{k}_i \mathbf{w}^\top$ is an $r \times r$ matrix. The coupling logit between positions j and k is:

$$\text{logits}_{jk} = \underbrace{\mathbf{q}_j^\top \mathbf{k}_k}_{\text{base}} + \underbrace{\mathbf{q}_j^\top W_j \mathbf{k}_k}_{\text{warp}} \quad (25)$$

where $\mathbf{q}_j = A^\top \mathbf{h}_j$ and $\mathbf{k}_k = B^\top \mathbf{h}_k$. The warp modifies the inner geometry: the effective coupling in r -space is $(I + W_j)$ instead of I .

B Abelian Attention: Sandpile Dynamics on the Coupling Field

All previous designs share a common structure: compute the coupling field (once or per-layer), then read from it. The computation is in the READING, not in the field itself. The field is passive—a lookup table that the response heads query.

The abelian sandpile model [36] suggests a radically different structure. In the sandpile, the field is an active dynamical system:

1. **Drive:** add one grain of sand (one token perturbs the field).
2. **Avalanche:** perturbations cascade through the system according to a local toppling rule, until the field reaches a new equilibrium.
3. **Read:** the equilibrium state is the representation.

The computation IS the avalanche—the cascading relaxation that propagates the perturbation through the geometry. The depth of computation is not L (a fixed number of layers) but the **cascade length**—how many toppling steps occur before equilibrium. This is adaptive: some tokens trigger massive avalanches (deep computation), others barely disturb the field (shallow computation).

The toppling rule is the operator. $\mathbf{M}$ defines how perturbations propagate. When a position’s coupling energy exceeds a learned threshold, it “topples”—redistributing energy to coupled positions according to $\mathbf{M}$ ’s geometry. The redistribution is **nonlinear** (only above-threshold positions topple), which prevents over-smoothing. Sub-threshold positions are stable and retain their coupling.

The abelian property is composition. In the abelian sandpile, the final state depends on *what* was added, not the *order* of addition. For language: the coupling field after processing tokens $\{A, B, C\}$ is the same regardless of processing order. This is composition in the strongest sense—the field is a function of the token SET, computed incrementally through cascades.

Self-organised criticality. The sandpile naturally evolves to a **critical state**—poised at the edge where small perturbations can cause cascades of any size. If the coupling field self-organises to criticality during training, then:

- Function words (“the,” “a”) cause small cascades (the field is already well-configured for common patterns).
- Content words (“vindicate,” “despite”) cause large cascades (the field must reorganise to accommodate new relational structure).
- The cascade length IS a measure of the token’s relational complexity.

Relation to previous designs. The abelian attention subsumes the multi-hop design: a cascade through k positions in the coupling graph IS a k -hop computation. But it adds nonlinearity (thresholding), adaptivity (cascade length varies), and composition (abelian property). The coupling field is not recomputed from scratch (transformer), not computed once and carried forward (our failed designs), but **incrementally evolved** through dynamical relaxation.

Implementation. The field is an $S \times d$ persistent state. At each token, the field is driven (token embedding added) and relaxed (iterated toppling through $\mathbf{M}$ ’s geometry until stable). Relaxation recomputes the coupling $C = \text{softmax}(\mathbf{hMh}^\top/\sqrt{d})$ at each step, so the coupling EVOLVES as the field relaxes—this is the cascade. Training uses backpropagation through the unrolled relaxation steps (or implicit differentiation at equilibrium, as in DEQ models).

Open questions.

- What is the precise toppling rule? The current design uses ReLU thresholding with $\mathbf{M}$ -weighted redistribution, but other rules (e.g., soft thresholding, attention-weighted redistribution) may work better.
- Does the field self-organise to criticality, or does it need to be trained toward it?
- Is the abelian property preserved during training, or does the nonlinear toppling break it?
- Can gradient descent learn a useful toppling geometry through $\mathbf{M}$?

This is the most speculative design in the paper. It is also the only one that has genuine adaptive computation, composition by construction, and a self-measuring diagnostic (cascade length = computational depth per token). Initial experiments showed the sandpile relaxation does not converge within the iteration budget. The idea remains theoretically interesting but the implementation needs refinement.

C Variants Under Test and Design Principles

Three meta-attention variants are under investigation, each implementing a different model of how the coupling field accumulates context. The goal is NOT to beat standard attention on val loss. The goal is to find an architecture that:

1. Is in the performance ballpark ($\pm 10\%$ of standard baselines).
2. Has **transparency**: we can inspect the grammar (base) and context (warp/field) separately, and each token’s contribution is attributable.
3. Has **composability**: the context representation composes algebraically (abelian accumulation or structured evolution through $\mathbf{M}$ ’s geometry).
4. Has **context compression**: the accumulated context is a fixed-size object (r^2 values) that summarises arbitrarily long history in coupling space.
5. Is **self-measuring**: the architecture’s own parameters (gates, decay rates, field norms) report how much each component contributes, without ablation.

Any architecture that satisfies (1) and demonstrates (2)–(5) is worth pursuing, even if it does not outperform a standard transformer on benchmarks. The innovation is in the *properties*, not the loss.

C.1 Variant A: Additive Warp (Tested)

$W_j = \sum_{i \leq j} \mathbf{w} \mathbf{k}_i \mathbf{w} \mathbf{v}_i^\top$ — each token contributes a rank-1 update, warps accumulate by addition.

- Composability: abelian by construction (addition commutes). Measured: 0.998 similarity under token reordering.
- Result: val 4.32, beating both base-only (4.75) and warp-only (4.47). Both components needed.
- Limitation: each token’s warp is context-independent — “vindicate” produces the same $\mathbf{w} \mathbf{k} \mathbf{w} \mathbf{v}^\top$ regardless of what came before. No iterative evolution.

C.2 Variant B: Iterative Field Evolution (Testing)

$W_t = \lambda(G W_{t-1} G^\top) + \mathbf{w} \mathbf{k}_t \mathbf{w} \mathbf{v}_t^\top$ — the field evolves through $\mathbf{M}$ ’s inner geometry $G = B^\top A$ at each token.

- Composability: not strictly abelian (the transition $GWG^\top$ is non-commutative), but the field evolution is structured through $\mathbf{M}$ ’s spectral geometry.
- The field state W_t reflects the accumulated *transformation* of the coupling by all previous tokens — not just their sum. Token 50’s effect depends on the field shaped by tokens 1–49.
- Self-measuring: the learned decay λ reports how fast context fades. The field norm $\|W_t\|$ reports how much context has accumulated. Both are inspectable during inference.
- Honest critique: if flattened, this is an SSM with Kronecker-structured transition ($G \otimes G$) on an r^2 -dimensional state. It is sequential. But the differences from a standard SSM are real: the state is a *coupling matrix* (not a feature vector), it feeds into *attention logits* (not direct output), and each mode of G interacts with others through the Kronecker structure (not diagonal).
- **Variant B2 (co-evolution):** The drive comes from the *attention output*, not the raw token:

$$\mathbf{a}_t = C_t V_t \quad (\text{what attention gathered at position } t) \quad (26)$$

$$\mathbf{w} \mathbf{k}_t, \mathbf{w} \mathbf{v}_t = \text{project}(\mathbf{a}_t) \quad (27)$$

$$W_t = \lambda G W_{t-1} G^\top + \mathbf{w} \mathbf{k}_t \mathbf{w} \mathbf{v}_t^\top \quad (28)$$

The field evolves based on what the model FOUND RELEVANT, not what tokens are present. This creates a feedback loop: the field shapes attention ($W \rightarrow C$), and attention shapes the field ($C \rightarrow W$). This is NOT an SSM — the “input” to the recurrence is the attention output, which itself depends on the field state. The field and the attention co-evolve.

- Mamba suffers because it *replaces* attention with recurrence. We *keep* attention but borrow the SSM’s context compression dynamics. The field evolves like an SSM state, but feeds INTO attention rather than replacing it. Attention quality + SSM compression.
- The diagnostic we watch: does the field grow with position (context accumulates) or decay (G kills it)? And in the co-evolution variant: does the attention-driven field produce richer dynamics than the token-driven field?
- **Connection to TTT.** The evolving field IS the test-time training channel, but without gradients. TTT updates correction weights via backward pass (expensive). The field updates through forward-pass operator dynamics (cheap at inference). The “learning rule” is baked into G during training; at inference, the field just evolves forward. This implements the three-timescale architecture: $\mathbf{M}_{\text{base}}$ (geological), FFN (trained), field W (inference-time, per-context).
- **Training cost (the trap).** At inference: no backward pass. But at *training*: gradients must flow through the entire recurrence (S sequential steps of BPTT through G). At $S = 128$: manageable. At $S = 2048$: the RNN tax. The co-evolution variant adds a second gradient path (attention $\rightarrow$ drive $\rightarrow$ field), making training harder to stabilise. Mitigations: truncated BPTT, periodic gradient detachment, or diagonal G (sacrificing mode interaction for parallelisability). This is the same trade-off Mamba makes: recurrence is efficient at inference but expensive to train.

C.3 Variant C: Abelian Attention (Tested, Failed)

Sandpile-style iterative relaxation of the coupling field. Cascading toppling through $\mathbf{M}$ ’s geometry until equilibrium.

- Result: val 5.05 vs 4.76 control. Worse. The field never reached equilibrium within the iteration budget (cascade length stuck at maximum).
- The toppling threshold did not adapt during training. The dynamics ran at full depth on every input without converging.

- The idea (adaptive computation through cascades, self-organised criticality) is theoretically appealing but the implementation does not work as designed.

What we are looking for. Not which variant gets the lowest loss. We are looking for a variant where:

- The field dynamics are **interpretable** — different inputs produce visibly different field states.
- The context **accumulates meaningfully** — later positions carry richer field structure than early positions.
- The base grammar and context warp are **both necessary** — neither alone suffices (already demonstrated for additive warp).
- The architecture **measures itself** — decay, gates, and field norms report on the computation during normal inference.

If any variant satisfies these while remaining in the performance ballpark, it is a viable foundation for the programme — an architecture that is both a model and a probe of relational computation.

D Two Fields: Cold Geometry and Hot Context

The architecture has two fields operating simultaneously at different timescales.

The cold field ($\mathbf{M}$). The coupling geometry. Learned slowly through gradient descent. Changes only during training. At inference, $\mathbf{M}$ is frozen. It IS the learned knowledge—the relational grammar, the structural invariants, the “muscle memory” of how language works.

The hot field (W). The context state. Evolves forward through operator dynamics as tokens are consumed. Changes at every token, **both during training and at inference**. It IS the active comprehension—the accumulated relational structure of what has been read.

The invariance property. The hot field works **identically at training and inference**. There is no “training mode” for W . The model is always doing the same thing: building context through the field as it reads. Training adds the slow $\mathbf{M}$ update; inference does not. But the hot field mechanism is the same.

	Training	Inference
Cold field ($\mathbf{M}$)	Updated by SGD	Frozen
Hot field (W)	Evolves forward (identical)	Evolves forward (identical)
Predictions	From $\mathbf{M} + W$	From $\mathbf{M} + W$

Training as meta-learning. Gradient descent does not train attention directly. It trains $\mathbf{M}$ to be a geometry that makes the hot field *work well*:

1. W evolves through tokens via the recurrence $W_t = \lambda G W_{t-1} G^\top + \text{drive}(x_t)$.
2. Predictions come from $\mathbf{M}_{\text{base}} + W$ injected into the coupling.
3. The loss measures how well $\mathbf{M} + W$ predicted.
4. SGD adjusts $\mathbf{M}$ so that W ’s dynamics improve.

The gradient through W tells $\mathbf{M}$: “when the hot field evolved this way through your geometry G , predictions at position 100 were wrong—adjust G so the field evolves better.”

This is meta-learning: the outer loop (SGD on $\mathbf{M}$) optimises the inner loop (W evolution through tokens). Like MAML, but the inner loop uses operator dynamics, not gradients—cheaper, structured, and identical at train and test time.

What good $\mathbf{M}$ looks like. Through training, $\mathbf{M}$ learns to define a geometry where:

- Important structural patterns persist (eigenmodes with $|\lambda_i| \approx 1$).
- Transient details fade (eigenmodes with $|\lambda_i| \ll 1$).
- Different relational templates rotate at different frequencies ($\arg(\lambda_i)$ varies).
- The hot field naturally compresses context into useful coupling structure.

The hot field is the PROBE of $\mathbf{M}$ ’s quality. Good $\mathbf{M} \rightarrow$ good field dynamics $\rightarrow$ good predictions. Bad $\mathbf{M} \rightarrow$ bad dynamics $\rightarrow$ bad predictions $\rightarrow$ gradient fixes $\mathbf{M}$.

Why this differs from a transformer. In a standard transformer, there is no “hot field.” The KV cache at inference is structurally different from training (training sees the whole sequence in parallel via causal masking; inference builds token by token). The mechanism simulates autoregressive processing but is not genuinely sequential.

In our design, the hot field is genuinely sequential—it evolves token by token through operator dynamics. Training and inference use the same mechanism. No simulation, no masking approximation. The model is always doing the real thing.

Experimental test. The invariance property is directly testable:

1. Train the model (both fields active, SGD updates $\mathbf{M}$).
2. At convergence, freeze $\mathbf{M}$.
3. Run the hot field on held-out data.
4. Compare: does the hot field produce the same quality when $\mathbf{M}$ is frozen (inference) as when $\mathbf{M}$ is being updated (training)?

If quality is the same: the hot field is truly invariant between training and inference. If quality degrades: the hot field was secretly depending on the gradient signal, and the invariance is broken.

A stronger test: freeze $\mathbf{M}$ from one dataset, run the hot field on a *different* dataset. If the hot field transfers: $\mathbf{M}$ learned a universal geometry. If it fails: $\mathbf{M}$ was dataset-specific. This is the frozen- $\mathbf{M}$ transfer experiment—already tested at +0.73% gap in the ORN, and now applicable to the spectral field architecture.

E On Depth: Criticality, Toppling, and Adaptive Computation

The operator programme began by trying to replace depth with algebra: compute $p(\mathbf{M})$ instead of L layers. This failed because depth provides multi-level indirection—each layer’s routing decisions depend on previous layers’ processing. The correct response is not to abandon depth but to understand what depth *does* in operator terms.

E.1 Depth as Height in a Sandpile

The abelian sandpile model provides the right metaphor, but applied to the **vertical** dimension (layers) rather than the horizontal (tokens within a layer).

Each layer adds “processing height” to each position. Most layers are *quiet*—the representation changes gradually, coupling barely shifts. Occasionally, accumulated processing crosses a threshold and the coupling *reorganises* dramatically. This is a toppling event: the relational structure undergoes a phase transition. The reorganisation cascades through positions via attention.

We already have evidence for this. The Krylov per-layer data shows:

Layers	Correction/displacement	Interpretation
1–4	4–8%	Sub-critical: quiet accumulation
5	16%	Approaching criticality
6–8	24–50%	Super-critical: toppling events

The transition at layers 4–5 is the critical point. Below it: the representation accumulates local context through $\mathbf{M}$ ’s coupling geometry. Above it: the accumulated information triggers reorganisation of the coupling structure. The correction gate ($r = -0.95$ with attention entropy) IS the threshold detector: it opens precisely when the field needs reorganisation.

E.2 Same Operator, Two Modes

$\mathbf{M}$ plays a dual role determined by the state height:

- **Below threshold (accumulation):** $\mathbf{M}$ gathers local context. Coupling is gentle. The warp adds context incrementally. Corrections are small.
- **Above threshold (reorganisation):** $\mathbf{M}$ propagates a restructuring cascade. The coupling pattern changes significantly. Corrections are large. The attention field undergoes a phase transition.

Same operator, different mode. The mode is determined by the accumulated processing, not by a per-layer parameter. This is why SharedM works: the geometry IS the same at both modes—it defines both how to gather and how to reorganise.

E.3 The Warp as Height

The context warp W carries across layers as the “sand height”:

$$W_l = W_{l-1} + \text{layer_drive}(\mathbf{h}_l) \quad (\text{accumulate height}) \quad (29)$$

$$\text{if } \|W_l\| > \theta : \quad \text{topple}(\mathbf{h}_l, W_l, \mathbf{M}) \quad (\text{reorganise}) \quad (30)$$

Each layer adds to the warp (quiet accumulation). When the warp exceeds a learned threshold θ , the coupling reorganises through $\mathbf{M}$ ’s geometry (toppling). After toppling, the warp relaxes (the accumulated “sand” is redistributed).

The number of toppling events is **adaptive**: simple inputs trigger few (the field stabilises early), complex inputs trigger many (multiple reorganisation cascades are needed). This gives variable-depth computation—not from a halting probability (Universal Transformer) but from **self-organised criticality**: the field naturally evolves to a state where toppling occurs when and where it is needed.

E.4 Composability Across Depth

Within each layer: the warp accumulates abelianly across tokens (horizontal composability, as before).

Across layers: the warp accumulates toward toppling thresholds (vertical composability through criticality). The composition of relational templates happens through toppling events: “the committee” is accumulated at layers 1–3, “vindicate” adds critical height at layer 4, the toppling at layer 5 reorganises the coupling into a three-party judgment structure. The composition is not algebraic (polynomial of $\mathbf{M}$). It is **dynamical**—driven by the accumulation and release of relational information through a critical system.

E.5 The Architecture

1. Stack L layers with SharedM.
2. Each layer has the spectral warp (context compression within the layer) from Section A.
3. The warp CARRIES ACROSS LAYERS—it is the accumulated processing height.
4. Each layer has a learned toppling threshold θ_l .
5. When the warp exceeds θ_l : the coupling reorganises (large correction, attention pattern changes significantly).
6. When below threshold: quiet accumulation (small correction, coupling barely changes).
7. Adaptive compute: stop adding layers when no position’s warp exceeds any threshold.

What this tests. The experiment trains two variants:

- **Carry-warp**: the warp persists across layers, accumulating toward toppling.
- **Reset-warp**: the warp resets at each layer (the standard approach—each layer is independent).

If carry-warp outperforms reset-warp: the vertical accumulation toward criticality adds value. The sandpile-over-depth model is supported. If not: each layer’s context compression is independent and the sandpile metaphor doesn’t extend to depth.

The deeper test: does the learned threshold θ_l vary across layers in a way consistent with the Krylov data? If θ is low at deep layers (easy to topple) and high at shallow layers (hard to topple): the architecture has learned that deep layers are where reorganisation happens. This would be the sandpile self-organising to the critical state we observed empirically.

F Beyond the Operator Programme: What We Got Wrong

The operator programme tried to replace depth with algebra. Every attempt failed—not because the idea of relational computation is wrong, but because we used the wrong tools. Every architecture we built was a transformer variant with a shared matrix. We applied polynomials, eigendecompositions, spectral rotations, and measured val loss. This is *left hemisphere* thinking applied to a *right hemisphere* problem.

The operator programme says: we must learn metaphor, and we must move by composition in abstract relational space. “Vindicate” does not activate eigenmode 5. It MAPS a relational template from one context to another—preserving three-party judgment structure while changing the specific parties. We looked for this in matrix spectra. We should have been looking for it in the **interaction between two different modes of processing**.

Three things we got wrong.

1. **The training objective.** Cross-entropy on next-token prediction is maximally literal: predict the specific next symbol. Every architecture we tested was optimised for this loss, so of course it converges to literal prediction machinery (FFN stores facts, attention routes tokens). To learn relational structure, we may need a fundamentally different pressure.
2. **The operator as a thing.** We kept trying to find the operator as a matrix, a polynomial, a spectral filter. But the brain doesn’t have an explicit “metaphor matrix.” It has two hemispheres whose interaction produces the capacity for metaphorical reasoning. The operator may be an *emergent property* of the right interaction, not a designed component.
3. **Measuring the wrong thing.** Val loss measures literal prediction. The operator’s value is richer representation: transfer, composition, generalisation, analogical reasoning. We never measured these.

This does not mean the programme failed—it means it hasn’t started. The next section outlines where it goes from here: a fundamentally different architecture inspired by the biological existence proof.

G Speculation: Reading a Book into the Geometry

Everything in this appendix is speculative. None of it has been tested. We include it because it follows naturally from the framework and may guide future experiments.

G.1 The Problem with Stored Context

A transformer reading a 100,000-token novel stores every token in a KV cache: $S \times L \times d$ values, growing linearly with context. Its “understanding” of the novel IS the stored tokens. Remove the cache and the model knows nothing. The memory is the comprehension.

But a human reading the same novel does not store 100,000 words. You build a world—characters, relationships, tensions, settings, narrative arcs. When asked “what happens next?”, you do not search through stored words. You consult your world model and generate what must follow from the structure.

G.2 Context as Geometry Warping

In the operator framework, reading a book would not fill a cache. It would warp the coupling geometry.

The base grammar $G = B^\top A$ is frozen—the relational structure learned during training. But as you read, the *effective* geometry changes:

$$G_{\text{eff}} = G + \Delta(\text{context}) \quad (31)$$

Each word read adds a small update. “The butler” activates person-role templates. “Had a motive” warps the butler↔crime coupling. “Was seen near the library” warps the butler↔location coupling. The grammar (G) has not changed. The world model ($G + \Delta$) has.

The update mechanism for each token x_t :

$$\mathbf{u}_t, \mathbf{v}_t = \text{project}(x_t) \quad (32)$$

$$\alpha_t = \text{gate}(x_t, \Delta_{t-1}) \quad (33)$$

$$\Delta_t = \Delta_{t-1} + \alpha_t \mathbf{u}_t \mathbf{v}_t^\top \quad (34)$$

Each token contributes a rank-1 update to the coupling geometry. Over T tokens, the geometry accumulates:

$$G_T = G + \sum_{t=1}^T \alpha_t \mathbf{u}_t \mathbf{v}_t^\top \quad (35)$$

This is structurally identical to the fast-weight update in linear attention ($W_t = W_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$). But the object being updated is not a hidden state or a KV cache. It is the **coupling geometry itself**—the relational structure of the world model.

G.3 Fixed-Size World Model

The KV cache stores $S \times L \times d$ values and grows linearly with context. The geometry $G_{\text{eff}} \in \mathbb{R}^{r \times r}$ is fixed-size regardless of context. At $r = 64$: 4,096 values store the relational structure of an arbitrarily long document.

This is possible because the geometry stores *relationships*, not tokens. After reading a mystery novel, you do not need “the word at position 47,392 was ‘butler.’” You need “the butler is suspected, has a motive, was near the library”—three relational links, three rank-1 updates. The specific positions do not matter. The structure does.

Old relationships that conflict with new ones are overwritten through competing rank-1 updates. The geometry self-organises as you read: early impressions are reinforced or revised by later evidence. This is how comprehension works—you do not remember everything, but you remember the relationships that matter.

G.4 Generation from the Warped Geometry

After reading 100K tokens, the effective geometry G_T reflects the accumulated world model. Generating the next token:

1. Query: $\mathbf{q} = A^\top \mathbf{h}_{\text{current}}$ (what am I asking?)
2. Depth: $c_k = g(\mathbf{h}_{\text{current}})$ (how deeply to probe?)
3. Probe: $\mathbf{q}_w = \sum c_k G_T^k \mathbf{q}$ (traverse the warped geometry)
4. Couple: $C = \mathbf{q}_w \mathbf{k}^\top$ (who relates to whom now?)
5. Respond: value gather, FFN, readout.

The polynomial of G_T composes the base grammar with the accumulated context. Degree 1: surface relationships (“the butler was mentioned”). Degree 3: nested inferences (“the butler’s motive connects to the inheritance which connects to the will”). Higher degrees traverse longer relational chains through the world model.

G.5 The Brain Analogy

When you read a novel, you build a world model. The world model is not a transcript—it is a web of relationships. The grammar of English is fixed (learned in childhood, crystallised). The world of the story is what you build by reading—a geometry warped by every sentence.

When you imagine what happens next, you traverse this geometry. The traversal depth depends on the question: “what colour was the house?” is shallow (degree 1, direct recall), while “who benefits from the murder?” is deep (degree 4+, multi-hop relational inference through motives, opportunities, and evidence).

The infinite context is not infinite memory. It is an **infinitely refineable geometry of finite size**. Each token refines it slightly. It never overflows. It never forgets a relationship that was reinforced. It gradually overwrites relationships that were contradicted.

This is how the operator programme, if successful, would handle context: not by storing what was read, but by absorbing it into the coupling geometry from which any necessary state can be recovered on demand.

Inference replaces memory.